



Uniwersytet Pomorski w Słupsku

Instytut Nauk Ścisłych i Technicznych

Krystian Górnowicz

**Jak obsłużyć proces wynajmu pojazdów przez osoby
prywatne**

Praca inżynierska

Numer albumu	50530
Kierunek	Informatyka
Specjalność	Programowanie aplikacji webowych
Profil studiów	Praktyczny
Data zakończenia pracy	13.09.2026
Promotor	mgr inż. Jakub Józef Bilski
Podpis promotora	

Spis treści

Wprowadzenie	5
1. Analiza problemu i założenia projektu	7
Cel i główne założenia.....	7
Charakterystyka problemu.....	8
Użytkownicy systemu.....	10
Wymagania funkcjonalne	10
Wymagania niefunkcjonalne	11
Przegląd i porównanie istniejących rozwiązań.....	12
Elementy wyróżniające projekt CarRental	13
2. Opis projektu	15
Ogólna koncepcja systemu	15
Aktorzy systemu i przypadki użycia	16
Proces rezerwacji pojazdu	17
Projekt architektury systemu	18
Projekt modelu danych	19
Projekt mechanizmów bezpieczeństwa	20
Podsumowanie projektu systemu	20
3. Implementacja systemu CarRental	21
Organizacja projektu i wykorzystane technologie.....	21
Zastosowane technologie.....	22
Implementacja obsługi użytkowników	22
Implementacja modułu samochodów	24
Prezentacja ofert i filtrowanie samochodów	26
Widok szczegółów pojazdu	28
Implementacja procesu rezerwacji	29
Potwierdzenie rezerwacji.....	30
Obsługa statusów	31
Implementacja mechanizmu dostępności	32
Implementacja opinii użytkowników	32

Implementacja powiadomień	33
Generowanie dokumentów związanych z wynajmem	34
Generowanie numeru faktury	36
Panel administratora	37
Implementacja mechanizmów bezpieczeństwa	40
Kontrola właściciela zasobu	40
Ochrona przed CSRF	41
Walidacja danych	41
Interfejs użytkownika	41
Wdrożenie aplikacji	43
Problemy napotkane podczas implementacji i wdrożenia	44
Podsumowanie implementacji	46
4. Testowanie i wyniki	46
Cel i zakres testów	47
Środowisko testowe	48
Organizacja testów automatycznych	49
Testowanie poszczególnych modułów	50
Wyniki przeprowadzonych testów	51
Problemy wykryte podczas testowania	52
Znaczenie testów bezpieczeństwa i kontroli dostępu	53
Ocena rezultatów testowania	53
Wnioski z testowania	54
5. Podsumowanie i wnioski	54
Podsumowanie realizacji projektu	54
Ocena zastosowanych technologii	55
Wnioski wynikające z realizacji projektu	56
Ograniczenia aktualnej wersji systemu	56
Możliwości dalszego rozwoju	57
Wnioski końcowe	58
Bibliografia	59
Linkografia	59
Spis rysunków	59

Spis tabel	61
Załączniki.....	62
Załącznik A. Nośnik CD - zawartość	62

Wprowadzenie

Przedmiotem niniejszej pracy jest zaprojektowanie, implementacja oraz wdrożenie aplikacji webowej wspomagającej proces wynajmu pojazdów przez osoby prywatne. Tematyka pracy została wybrana ze względu na rosnącą potrzebę cyfryzacji usług związanych z udostępnianiem i wynajmem samochodów. W przypadku obsługi większej liczby pojazdów konieczne staje się uporządkowanie informacji dotyczących właścicieli, najemców, terminów rezerwacji, dostępności pojazdów, dokumentów oraz rozliczeń. Brak jednego spójnego systemu może prowadzić do powstawania konfliktów terminów, utrudniać kontrolę nad rezerwacjami oraz zwiększać ryzyko błędów organizacyjnych.

Celem pracy było stworzenie działającego systemu internetowego umożliwiającego obsługę najważniejszych etapów procesu wynajmu pojazdu. Opracowana aplikacja CarRental pozwala użytkownikom na przeglądanie dostępnych samochodów, filtrowanie ofert według wybranych parametrów, sprawdzanie szczegółowych informacji o pojeździe oraz składanie rezerwacji na określony termin. Właściciel pojazdu może zarządzać własnymi samochodami oraz obsługiwać zgłoszenia dotyczące ich wynajmu, natomiast administrator otrzymuje dostęp do funkcji umożliwiających nadzorowanie użytkowników, pojazdów, rezerwacji oraz pozostałych elementów systemu.

Istotnym założeniem projektu było przygotowanie kompletnej aplikacji, a nie jedynie prototypu lub warstwy prezentacyjnej. System obejmuje mechanizmy rejestracji i logowania użytkowników, zarządzania profilem, dodawania i edycji pojazdów, obsługi zdjęć, wyszukiwania i filtrowania samochodów, rezerwowania pojazdów, zmiany statusów rezerwacji, generowania dokumentów oraz faktur, obsługi opinii i powiadomień, a także funkcje administracyjne. Zastosowano również mechanizmy kontroli dostępu pozwalające rozdzielić możliwości dostępne dla zwykłego użytkownika, właściciela pojazdu i administratora.

Aplikacja została wykonana z wykorzystaniem języka PHP oraz frameworka Symfony 6.4. Do przechowywania danych zastosowano relacyjną bazę MySQL oraz bibliotekę Doctrine ORM, odpowiedzialną za mapowanie encji aplikacji na strukturę bazy danych. Warstwa prezentacji została przygotowana przy użyciu systemu szablonów Twig oraz biblioteki Bootstrap. Do obsługi logowania, autoryzacji i kontroli uprawnień wykorzystano komponent Symfony Security. Projekt został następnie wdrożony na serwerze AWS EC2 działającym pod kontrolą systemu Linux, gdzie żądania HTTP obsługiwane są przez serwer nginx oraz PHP-FPM. Dzięki temu możliwe było zweryfikowanie działania aplikacji również poza lokalnym środowiskiem programistycznym.

W projekcie szczególną uwagę zwrócono na bezpieczeństwo danych oraz kontrolę dostępu do poszczególnych funkcji. Hasła użytkowników nie są przechowywane w postaci jawnej, a dostęp do chronionych zasobów zależy od przypisanych ról i aktualnie uwierzytelnionego użytkownika. Operacje zmieniające dane mogą być dodatkowo chronione mechanizmem tokenów CSRF. Dane konfiguracyjne, takie jak hasła do bazy danych oraz inne informacje środowiskowe, są przechowywane poza publicznym repozytorium kodu. W celu ograniczenia zakresu przetwarzania danych finansowych aplikacja nie obsługuje bezpośrednich płatności elektronicznych i nie przechowuje danych kart płatniczych.

Zakres pracy obejmuje analizę problemu, określenie wymagań funkcjonalnych i нефункциональных, zaprojektowanie struktury systemu oraz modelu danych, implementację najważniejszych modułów, wdrożenie aplikacji w środowisku produkcyjnym oraz przeprowadzenie testów jej działania. Poza zakresem aktualnej wersji projektu pozostawiono między innymi bezpośrednią integrację z operatorem płatności elektronicznych. Element ten został potraktowany jako możliwy kierunek dalszego rozwoju systemu.

Praca została podzielona na pięć głównych rozdziałów. W pierwszym rozdziale przedstawiono analizę problemu, cel projektu, główne założenia oraz przegląd istniejących rozwiązań. Drugi rozdział zawiera opis projektowanego systemu, jego wymagania oraz główne założenia funkcjonalne i techniczne. W trzecim rozdziale przedstawiono implementację aplikacji, zastosowane technologie, architekturę, model danych, wybrane fragmenty kodu źródłowego oraz proces wdrożenia. Rozdział czwarty poświęcono testowaniu systemu i analizie uzyskanych wyników. W ostatnim rozdziale podsumowano wykonane prace, oceniono stopień realizacji założonych celów oraz wskazano możliwości dalszej rozbudowy aplikacji.

1. Analiza problemu i założenia projektu

Cel i główne założenia

Celem pracy jest zaprojektowanie, implementacja oraz wdrożenie aplikacji webowej CarRental wspomagającej proces wynajmu pojazdów przez osoby prywatne. System został zaprojektowany w taki sposób, aby umożliwić obsługę najważniejszych etapów związanych z udostępnieniem samochodu, jego wyszukaniem, rezerwacją oraz późniejszą obsługą wynajmu.

Proces wynajmu pojazdu wymaga wymiany i przechowywania wielu informacji dotyczących między innymi użytkowników, samochodów, terminów, cen oraz dokumentów. W przypadku braku jednego systemu dane te mogą być przechowywane w różnych miejscach, takich jak wiadomości e-mail, komunikatory, dokumenty tekstowe lub arkusze kalkulacyjne. Utrudnia to kontrolowanie aktualnego stanu rezerwacji i dostępności samochodów oraz zwiększa ryzyko wystąpienia pomyłek.

Szczególnie istotnym problemem jest kontrolowanie terminów wynajmu. Ten sam samochód nie powinien zostać udostępniony kilku osobom w nakładających się terminach. Konieczne jest zatem powiązanie informacji o pojeździe z jego rezerwacjami oraz sprawdzanie dostępności przed utworzeniem kolejnego zgłoszenia. Równie ważne jest umożliwienie właścicielowi pojazdu podejmowania decyzji dotyczącej jego udostępnienia.

Opracowana aplikacja CarRental ma za zadanie uporządkować wymienione procesy poprzez wykorzystanie centralnej bazy danych oraz jednolitego interfejsu dostępnego z poziomu przeglądarki internetowej. Użytkownik może utworzyć konto, przeglądać oferty pojazdów, korzystać z mechanizmu filtrowania, sprawdzać szczegóły wybranego samochodu oraz dokonać jego rezerwacji. Osoba posiadająca pojazd może natomiast dodać go do systemu i obsługiwać zgłoszenia dotyczące wynajmu.

Założono, że CarRental będzie kompletną aplikacją webową, a nie wyłącznie prototypem interfejsu użytkownika. Projekt obejmuje warstwę prezentacji, logikę aplikacji, relacyjną bazę danych, mechanizmy uwierzytelniania i autoryzacji oraz środowisko produkcyjne. System powinien umożliwiać wykonanie pełnego podstawowego procesu rozpoczynającego się od utworzenia konta i dodania pojazdu, poprzez jego wyszukanie i zarezerwowanie, aż do obsługi zgłoszenia oraz dokumentów związanych z wynajmem.

Do głównych założeń projektu zaliczono:

- utworzenie centralnej bazy danych przechowującej informacje dotyczące użytkowników, samochodów i rezerwacji;
- umożliwienie rejestracji i logowania użytkowników;

- umożliwienie zarządzania danymi profilu;
- umożliwienie użytkownikom dodawania własnych pojazdów;
- obsługę zdjęcia głównego oraz galerii pojazdu;
- udostępnienie katalogu samochodów;
- umożliwienie filtrowania ofert między innymi według marki, modelu, roku produkcji, ceny, lokalizacji i dostępności;
- prezentowanie szczegółowych informacji o pojeździe;
- prezentowanie informacji dotyczących dostępności samochodu;
- obsługę procesu rezerwacji;
- kontrolowanie kolizji terminów;
- umożliwienie właścicielowi zaakceptowania lub odrzucenia zgłoszenia;
- obsługę statusów rezerwacji;
- generowanie dokumentów związanych z wynajmem;
- obsługę faktur;
- obsługę powiadomień;
- umożliwienie dodawania opinii o samochodach;
- zapewnienie panelu administracyjnego;
- zastosowanie mechanizmów kontroli dostępu do chronionych zasobów;
- wdrożenie aplikacji na serwerze produkcyjnym.

W aktualnej wersji projektu świadomie zrezygnowano z bezpośredniego przetwarzania płatności elektronicznych. Decyzja ta wynikała nie tylko z ograniczenia zakresu pracy, lecz również z wymagań formalnych i technicznych związanych z obsługą płatności online. Integracja z operatorem płatności wymagałaby obsługi statusów transakcji, zwrotów, reklamacji, potwierdzeń, bezpieczeństwa danych finansowych oraz zgodności z regulaminem wybranego dostawcy usług płatniczych. W przypadku płatności kartowych konieczne byłoby również uwzględnienie wymagań standardu PCI DSS, dotyczącego bezpieczeństwa danych kart płatniczych. Dlatego w aktualnej wersji aplikacja nie przechowuje danych kart, a rozliczenie pomiędzy stronami odbywa się poza systemem.

Charakterystyka problemu

Wynajem pojazdu pomiędzy osobami prywatnymi wymaga zapewnienia obu stronom dostępu do aktualnych i jednoznacznych informacji. Osoba zainteresowana wynajmem powinna mieć możliwość sprawdzenia podstawowych parametrów samochodu, jego

ceny, lokalizacji oraz dostępności. Właściciel pojazdu potrzebuje natomiast narzędzi pozwalających na publikowanie oferty i kontrolowanie zgłoszeń dotyczących wynajmu.

Jednym z podstawowych problemów jest zarządzanie dostępnością pojazdu. Wraz ze wzrostem liczby użytkowników oraz rezerwacji ręczne kontrolowanie terminów staje się coraz trudniejsze. Powstaje możliwość utworzenia kilku rezerwacji obejmujących ten sam samochód i nakładające się okresy. System powinien zatem uwzględniać istniejące rezerwy podczas wyboru kolejnego terminu.

Drugim istotnym zagadnieniem jest kontrola właściciela nad własnym pojazdem. Samo zainteresowanie samochodem przez innego użytkownika nie powinno oznaczać automatycznej zgody na jego udostępnienie. W CarRental rezerwaacja może zostać utworzona jako oczekująca, a właściciel otrzymuje możliwość jej zaakceptowania lub odrzucenia. Pozwala to zachować kontrolę nad procesem wynajmu.

Kolejnym problemem jest rozproszenie informacji związanych z wynajmem. Dane użytkownika, informacje o samochodzie, wybrany termin, aktualny status rezerwacji oraz dokumenty powinny być ze sobą powiązane. Wykorzystanie relacyjnej bazy danych pozwala na przechowywanie tych informacji w uporządkowanej formie oraz zachowanie zależności pomiędzy poszczególnymi elementami systemu.

Istotne znaczenie ma również komunikacja dotycząca zmian zachodzących w procesie. Użytkownik powinien mieć możliwość sprawdzenia stanu swojej rezerwacji, natomiast właściciel powinien otrzymywać informacje o zgłoszeniach dotyczących jego samochodów. W tym celu aplikacja wykorzystuje mechanizm powiadomień związanych z określonymi zdarzeniami.

System przetwarza również dane użytkowników, dlatego jednym z elementów projektu jest zapewnienie odpowiedniej kontroli dostępu. Funkcje wymagające uwierzytelnienia nie powinny być dostępne dla niezalogowanych osób, a operacje dotyczące konkretnego samochodu powinny być ograniczone do jego właściciela lub administratora posiadającego odpowiednie uprawnienia.

Opracowanie aplikacji wymaga zatem rozwiązania nie tylko problemów związanych z interfejsem użytkownika, ale również z przechowywaniem danych, kontrolą terminów, bezpieczeństwem, zarządzaniem uprawnieniami oraz wdrożeniem systemu w środowisku produkcyjnym.

Użytkownicy systemu

W systemie CarRental można wyróżnić użytkownika korzystającego z ofert, właściciela pojazdu oraz administratora. Podział ten wynika przede wszystkim z zakresu operacji wykonywanych przez poszczególne osoby.

Podstawowy użytkownik systemu może utworzyć konto, zalogować się, zarządzać danymi swojego profilu oraz przeglądać samochody udostępnione w serwisie. Może korzystać z mechanizmu filtrowania ofert, przechodzić do szczegółów wybranego samochodu, sprawdzać informacje dotyczące jego dostępności oraz tworzyć rezerwacje. Użytkownik otrzymuje również dostęp do informacji dotyczących własnych rezerwacji, powiadomień i dokumentów związanych z procesem wynajmu.

Właściciel pojazdu nie musi stanowić całkowicie oddzielnego rodzaju konta. Użytkownik może jednocześnie korzystać z samochodów innych osób oraz udostępniać własne pojazdy. Po dodaniu samochodu staje się jego właścicielem w kontekście systemu i otrzymuje możliwość wykonywania operacji dotyczących tego pojazdu.

Właściciel może dodawać samochody wraz z podstawowymi informacjami, ceną, lokalizacją oraz zdjęciami. Może również przeglądać zgłoszenia dotyczące własnych pojazdów oraz podejmować decyzję o zaakceptowaniu lub odrzuceniu rezerwacji. Takie rozwiązanie umożliwia jednemu użytkownikowi występowanie po obu stronach procesu wynajmu bez konieczności tworzenia dwóch oddzielnych kont.

Administrator posiada rozszerzone uprawnienia związane z nadzorowaniem działania całego systemu. Panel administracyjny umożliwia dostęp do danych dotyczących użytkowników, samochodów oraz rezerwacji i wykonywanie operacji administracyjnych niedostępnych dla zwykłego użytkownika.

Rozdzielenie zakresu dostępnych operacji stanowi jeden z podstawowych mechanizmów bezpieczeństwa aplikacji. Użytkownik powinien mieć możliwość wykonywania jedynie operacji, do których posiada odpowiednie uprawnienia.

Wymagania funkcjonalne

Na podstawie przeprowadzonej analizy określono wymagania funkcjonalne aplikacji. Definiują one operacje, które system powinien umożliwiać użytkownikom.

Tabela 1. Wymagania funkcjonalne systemu CarRental

Identyfikator	Wymaganie funkcjonalne
WF01	System umożliwia rejestrację użytkownika.
WF02	System umożliwia logowanie i wylogowanie użytkownika.
WF03	System umożliwia przeglądanie i edycję danych profilu.
WF04	System umożliwia przeglądanie listy samochodów.

WF05	System umożliwia filtrowanie samochodów według określonych parametrów.
WF06	System umożliwia wyświetlenie szczegółowych informacji o samochodzie.
WF07	System prezentuje informacje dotyczące dostępności samochodu.
WF08	System umożliwia utworzenie rezerwacji na określony termin.
WF09	System kontroluje możliwość wystąpienia kolizji terminów rezerwacji.
WF10	System umożliwia użytkownikowi przeglądanie własnych rezerwacji.
WF11	System umożliwia użytkownikowi dodanie własnego samochodu.
WF12	System umożliwia zarządzanie pojazdami należącymi do użytkownika.
WF13	System umożliwia właścicielowi przeglądanie rezerwacji dotyczących jego pojazdów.
WF14	System umożliwia właścicielowi zaakceptowanie lub odrzucenie rezerwacji.
WF15	System obsługuje statusy rezerwacji.
WF16	System umożliwia generowanie dokumentów związanych z wynajmem.
WF17	System obsługuje dokumenty rozliczeniowe związane z wynajmem.
WF18	System umożliwia dodawanie opinii dotyczących samochodów.
WF19	System obsługuje powiadomienia związane ze zdarzeniami dotyczącymi rezerwacji.
WF20	System udostępnia funkcje administracyjne dla użytkownika posiadającego odpowiednie uprawnienia.

Zastosowanie identyfikatorów wymagań pozwala na późniejsze jednoznaczne powiązanie poszczególnych funkcji z przypadkami testowymi. Ułatwia to ocenę, czy wszystkie założone elementy zostały poprawnie zaimplementowane.

Wymagania niefunkcjonalne

Wymagania niefunkcjonalne dotyczą właściwości systemu, które nie stanowią bezpośrednich funkcji dostępnych dla użytkownika, ale mają wpływ na bezpieczeństwo, możliwość utrzymania projektu i sposób jego działania.

Tabela 2. Wymagania niefunkcjonalne systemu CarRental

Identyfikator	Kategoria	Wymaganie
WNF01	Dostępność	Aplikacja powinna być dostępna z poziomu współczesnej przeglądarki internetowej.
WNF02	Dane	Dane systemu powinny być przechowywane w relacyjnej bazie danych.
WNF03	Bezpieczeństwo	Dostęp do chronionych funkcji powinien wymagać uwierzytelnienia.
WNF04	Bezpieczeństwo	Dostęp do funkcji administracyjnych powinien być ograniczony do użytkowników posiadających odpowiednie uprawnienia.
WNF05	Bezpieczeństwo	Hasła użytkowników nie mogą być przechowywane w bazie danych w postaci jawnej.
WNF06	Bezpieczeństwo	Operacje zmieniające dane powinny być zabezpieczone przed nieuprawnionym wykonaniem.
WNF07	Bezpieczeństwo	Dane konfiguracyjne i sekrety powinny być przechowywane poza publicznym repozytorium kodu.
WNF08	Utrzymanie	Struktura kodu powinna umożliwiać rozwijanie poszczególnych modułów aplikacji.
WNF09	Wdrożenie	System powinien mieć możliwość działania na serwerze Linux.

WNF10	Dane finansowe	Aplikacja nie powinna przechowywać danych kart płatniczych w aktualnej wersji projektu.
-------	----------------	---

Wymagania te zostały uwzględnione podczas wyboru technologii oraz projektowania architektury systemu. W szczególności istotne było zastosowanie mechanizmów bezpieczeństwa udostępnianych przez framework Symfony oraz rozdzielenie odpowiedzialności pomiędzy poszczególne komponenty aplikacji.

Przegląd i porównanie istniejących rozwiązań

Na rynku funkcjonują platformy internetowe umożliwiające krótkoterminowy wynajem oraz współdzielenie samochodów. Rozwiązania tego typu łączą właścicieli pojazdów z osobami zainteresowanymi wynajmem, zapewniając wyszukiwanie ofert, obsługę terminów, komunikację pomiędzy stronami oraz dodatkowe mechanizmy wspierające bezpieczeństwo transakcji.

Do najbardziej rozpoznawalnych przykładów takich rozwiązań należą Turo, Getaround oraz HoppyGo. Każda z tych platform realizuje podobną ogólną ideę, jednak różni się zakresem automatyzacji, sposobem obsługi płatności, modelem ochrony użytkowników oraz skalą działania.

Getaround umożliwia wyszukiwanie i wynajem samochodów udostępnianych przez innych użytkowników. Platforma kładzie nacisk na automatyzację procesu oraz możliwość samodzielnego odbioru pojazdu w wybranych przypadkach. Rozwiązanie Getaround Connect pozwala ograniczyć udział właściciela w przekazaniu samochodu, ponieważ część operacji może być wykonywana z poziomu telefonu.

Turo działa jako rozbudowana platforma pośrednicząca pomiędzy właścicielami samochodów a osobami, które chcą je wynająć. Oprócz publikowania ofert i obsługi rezerwacji platforma obejmuje mechanizmy związane z płatnościami, ochroną wynajmu, zasadami anulowania, ocenami użytkowników oraz dodatkowymi wymaganiami dotyczącymi uczestników procesu.

HoppyGo również opiera się na modelu wynajmu samochodów pomiędzy użytkownikami. System pośredniczy w publikowaniu ofert, wyborze terminu oraz obsłudze procesu wynajmu. Podobnie jak pozostałe platformy komercyjne, rozwiązanie to zawiera elementy formalnej i finansowej obsługi transakcji, które wykraczają poza zakres niniejszego projektu.

Wymienione rozwiązania są systemami komercyjnymi działającymi na większą skalę niż projekt CarRental. Obejmują one funkcje wymagające znacznie szerszego zaplecza organizacyjnego, takie jak obsługa płatności internetowych, polityki ochrony wynajmu, weryfikacja użytkowników, procedury reklamacyjne, regulaminy korzystania z pojazdów oraz, w niektórych przypadkach, rozwiązania telematyczne.

CarRental wykorzystuje podobną podstawową koncepcję, jednak zakres systemu został skoncentrowany na najważniejszych etapach procesu wynajmu pojazdu pomiędzy

osobami prywatnymi. Użytkownik może jednocześnie występować jako osoba wynajmująca pojazd oraz jako właściciel udostępniający własny samochód. Nie jest konieczne tworzenie oddzielnych kont przeznaczonych dla obu stron procesu.

Podstawowe różnice pomiędzy analizowanymi rozwiązaniami a CarRental przedstawiono w tabeli.

Tabela 3. Porównanie wybranych platform wynajmu pojazdów z systemem CarRental

Funkcja	Getaround	Turo	HoppyGo	CarRental
Rejestracja użytkowników	Tak	Tak	Tak	Tak
Udostępnianie pojazdów przez właścicieli	Tak	Tak	Tak	Tak
Wyszukiwanie samochodów	Tak	Tak	Tak	Tak
Wybór terminu wynajmu	Tak	Tak	Tak	Tak
Obsługa rezerwacji	Tak	Tak	Tak	Tak
Zarządzanie ofertą przez właściciela	Tak	Tak	Tak	Tak
Opinie użytkowników	Tak	Tak	Tak	Tak
Płatności internetowe w ramach platformy	Tak	Tak	Tak	Nie
Mechanizmy związane z ubezpieczeniem	Tak	Tak	Tak	Nie
Zaawansowana weryfikacja użytkownika	Tak	Tak	Tak	Nie
Zdalne otwieranie wybranych pojazdów	Tak	Nie jako podstawowa funkcja	Nie	Nie
Kontrola rezerwacji przez właściciela	Tak	Tak	Tak	Tak
Dokumenty generowane na podstawie danych rezerwacji	Zależne od platformy	Zależne od platformy	Zależne od platformy	Tak

Porównanie pokazuje, że CarRental realizuje podstawowy rdzeń procesu znany z komercyjnych platform: publikowanie pojazdu, wyszukiwanie oferty, wybór terminu, utworzenie rezerwacji oraz obsługę decyzji właściciela. Różnica polega na świadomym ograniczeniu zakresu projektu do części organizacyjnej i dokumentacyjnej. System nie zastępuje pełnej platformy komercyjnej, lecz stanowi działającą aplikację webową prezentującą najważniejsze mechanizmy potrzebne do obsługi wynajmu pojazdów.

Elementy wyróżniające projekt CarRental

Celem projektu CarRental nie jest bezpośrednie konkutowanie pod względem liczby funkcji z dużymi platformami komercyjnymi. System został zaprojektowany jako rozwiązanie skoncentrowane na podstawowym procesie wynajmu pojazdu pomiędzy osobami prywatnymi. Pozwala to ograniczyć liczbę dodatkowych etapów i skupić się na funkcjach bezpośrednio związanych z udostępnieniem samochodu, rezerwacją, decyzją właściciela oraz dokumentacją wynajmu.

Jednym z istotnych założeń CarRental jest możliwość występowania tego samego użytkownika po obu stronach procesu. Użytkownik może przeglądać i rezerwować samochody innych osób, a jednocześnie dodawać własne pojazdy i udostępniać je do wynajmu. Funkcja właściciela wynika z posiadania samochodu przypisanego do konta, dzięki czemu nie ma potrzeby zakładania osobnego konta przeznaczonego wyłącznie do publikowania ofert.

Istotnym elementem systemu jest również pozostawienie właścicielowi kontroli nad zgłoszeniami dotyczącymi jego pojazdu. Utworzenie rezerwacji nie musi automatycznie oznaczać udostępnienia samochodu. Zgłoszenie może otrzymać status oczekujący, a właściciel podejmuje decyzję o jego zaakceptowaniu lub odrzuceniu. Pozwala to zachować kontrolę nad prywatnym pojazdem, a jednocześnie uporządkować komunikację pomiędzy stronami.

Kolejnym elementem jest powiązanie rezerwacji z dokumentacją dotyczącą wynajmu. Dane użytkownika, samochodu i okresu rezerwacji przechowywane w systemie mogą zostać wykorzystane podczas generowania dokumentów. Ogranicza to konieczność ponownego ręcznego wprowadzania tych samych informacji do oddzielnych formularzy lub dokumentów.

CarRental integruje również informacje dotyczące przebiegu procesu wynajmu. Użytkownik posiada dostęp do swoich rezerwacji i ich statusów, natomiast właściciel może przeglądać zgłoszenia dotyczące jego samochodów. Mechanizm powiadomień umożliwia informowanie użytkowników o istotnych zdarzeniach związanych z rezerwacją. Informacje o samochodzie, użytkowniku, terminie, statusie oraz dokumentach pozostają dzięki temu powiązane w ramach jednego systemu.

W aktualnej wersji CarRental świadomie zrezygnowano z bezpośredniego przetwarzania płatności elektronicznych oraz danych kart płatniczych. Rozwiązanie takie ogranicza zakres danych finansowych przetwarzanych przez aplikację, zmniejsza ryzyko niebezpieczeństwa oraz pozwala skoncentrować projekt na mechanizmach bezpośrednio związanych z wynajmem: publikacji pojazdu, wyszukiwaniu ofert, wyborze terminu, rezerwacji, decyzji właściciela, dokumentach i powiadomieniach. Integracja płatności może zostać dodana w przyszłości, jednak wymagałaby zaprojektowania dodatkowego modułu obsługującego operatora płatności, webhooki, statusy transakcji, zwroty oraz wymagania bezpieczeństwa.

Wartością projektu CarRental nie jest zatem zastąpienie rozbudowanych platform komercyjnych, lecz zaprojektowanie i implementacja spójnego systemu dostosowanego do przyjętego scenariusza wynajmu pojazdów pomiędzy osobami prywatnymi. Aplikacja łączy w jednym środowisku publikację pojazdu, wyszukiwanie ofert, kontrolę dostępności, rezerwację, decyzję właściciela, obsługę statusów, dokumenty, powiadomienia oraz opinie użytkowników. Jednocześnie przyjęta architektura umożliwia dalszą rozbudowę systemu między innymi o płatności elektroniczne,

bardziej zaawansowaną weryfikację użytkowników, mechanizmy ubezpieczeniowe oraz dodatkową automatyzację procesu wynajmu.

2. Opis projektu

Ogólna koncepcja systemu

CarRental jest aplikacją webową przeznaczoną do wspomagania procesu wynajmu pojazdów pomiędzy osobami prywatnymi. System został zaprojektowany w architekturze klient-serwer, dzięki czemu użytkownik korzysta z jego funkcji za pośrednictwem przeglądarki internetowej, bez konieczności instalowania dodatkowego oprogramowania.

Podstawowym założeniem projektu jest zgromadzenie w jednym systemie informacji dotyczących użytkowników, pojazdów, rezerwacji oraz elementów powiązanych z procesem wynajmu. Dane są przechowywane w relacyjnej bazie danych, natomiast logika aplikacji odpowiada za ich przetwarzanie, kontrolę dostępu oraz realizację operacji wykonywanych przez użytkowników.

System umożliwia użytkownikowi przeglądanie ofert samochodów, wyszukiwanie pojazdów według określonych kryteriów oraz przechodzenie do szczegółowego widoku wybranego samochodu. Zalogowany użytkownik może dokonać rezerwacji pojazdu na określony przedział czasu, a następnie sprawdzać jej aktualny status.

Użytkownik może również udostępniać własne pojazdy. Samochód jest przypisywany do konta osoby, która dodała go do systemu. Właściciel uzyskuje możliwość zarządzania informacjami dotyczącymi pojazdu oraz obsługi rezerwacji składanych przez pozostałych użytkowników. Pozwala to na korzystanie z jednego konta zarówno w charakterze osoby wynajmującej samochód, jak i właściciela udostępniającego własny pojazd.

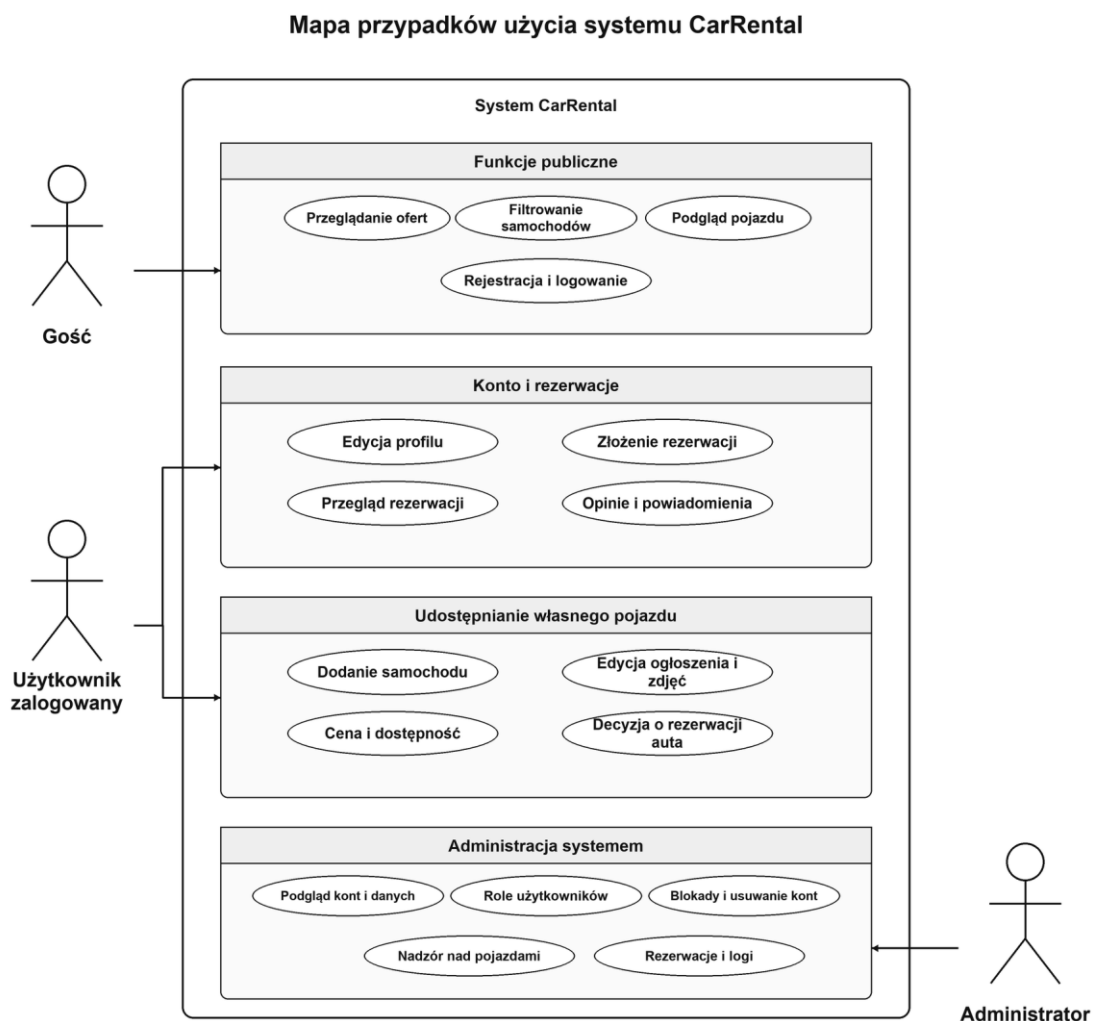
Dodatkową rolę w systemie pełni administrator posiadający rozszerzony zakres uprawnień. Funkcje administracyjne służą do nadzorowania danych znajdujących się w aplikacji oraz zarządzania jej podstawowymi zasobami.

Projekt obejmuje również mechanizmy związane z powiadomieniami, opiniami użytkowników oraz dokumentami dotyczącymi procesu wynajmu. Poszczególne moduły wykorzystują wspólny model danych, dzięki czemu informacje dotyczące określonego użytkownika, pojazdu i rezerwacji mogą pozostawać ze sobą powiązane.

Aktorzy systemu i przypadki użycia

W systemie CarRental można wyróżnić trzy podstawowe typy aktorów: gościa, użytkownika zalogowanego oraz administratora. Właściciel pojazdu nie stanowi odrębnego typu konta, ponieważ jest to zwykły użytkownik, który dodał samochód do systemu. Funkcje właścicielskie są więc częścią konta użytkownika i wynikają z relacji użytkownika do konkretnego pojazdu, a nie z osobnej rejestracji. Administrator jest kontem posiadającym rolę `ROLE_ADMIN`, które może zostać utworzone skryptem lub nadane istniejącemu użytkownikowi. Jego zadania obejmują nadzór nad kontami, nadawanie ról, blokowanie dostępu, kontrolę pojazdów i rezerwacji oraz przegląd logów systemowych.

Zakres funkcji przypisanych do poszczególnych ról przedstawiono w formie mapy przypadków użycia. Taka forma jest bardziej czytelna niż rozbudowany diagram z dużą liczbą przecinających się asocjacji, a jednocześnie pozwala jednoznacznie określić, które operacje są dostępne dla danego aktora.

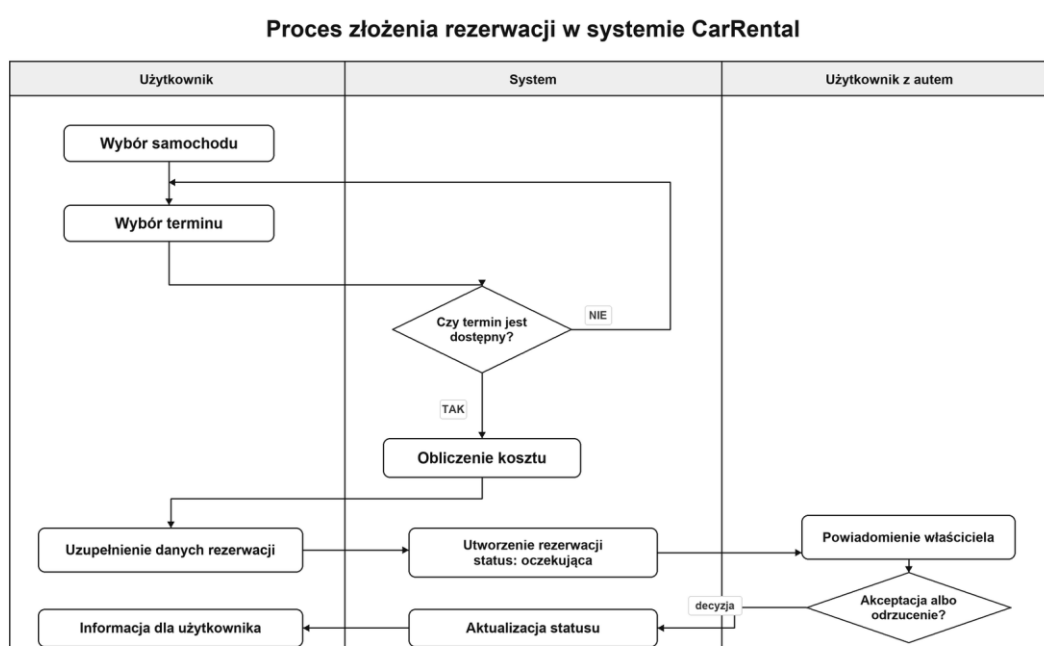


Rysunek 1. Mapa przypadków użycia systemu CarRental.

Mapa pokazuje, że jeden użytkownik może korzystać z aplikacji w dwóch kontekstach: jako osoba rezerwująca samochód oraz jako osoba udostępniająca własny pojazd. Każdy zalogowany użytkownik może dodać samochód do systemu, a następnie edytować własne ogłoszenie, zarządzać zdjęciami, określać dostępność oraz podejmować decyzje dotyczące rezerwacji tego samochodu.

Proces rezerwacji pojazdu

Najważniejszym procesem biznesowym aplikacji jest złożenie rezerwacji samochodu. Proces rozpoczyna się od wyboru pojazdu i terminu, a następnie obejmuje sprawdzenie dostępności, obliczenie kosztu, uzupełnienie danych kontaktowych oraz utworzenie rezerwacji oczekującej na decyzję użytkownika udostępniającego dany pojazd.

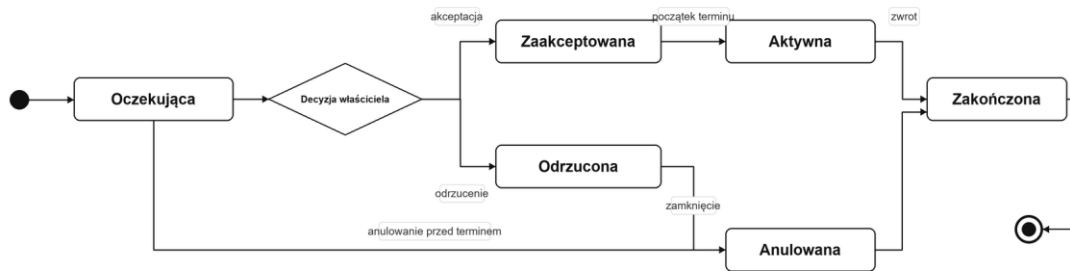


Rysunek 2. Diagram procesu rezerwacji pojazdu.

Diagram procesu porządkuje odpowiedzialności pomiędzy użytkownikiem składającym rezerwację, systemem i użytkownikiem udostępniającym pojazd. System odpowiada za sprawdzenie konfliktów terminów, obliczenie kosztu oraz zmianę statusu rezerwacji. Użytkownik z autem podejmuje decyzję o akceptacji albo odrzuceniu zgłoszenia, a osoba rezerwująca otrzymuje informację o wyniku tej decyzji.

Zmiana statusu rezerwacji ma wpływ na dalszą obsługę wynajmu, dlatego cykl życia rezerwacji przedstawiono dodatkowo jako osobny diagram stanów.

Cykl życia rezerwacji w systemie CarRental

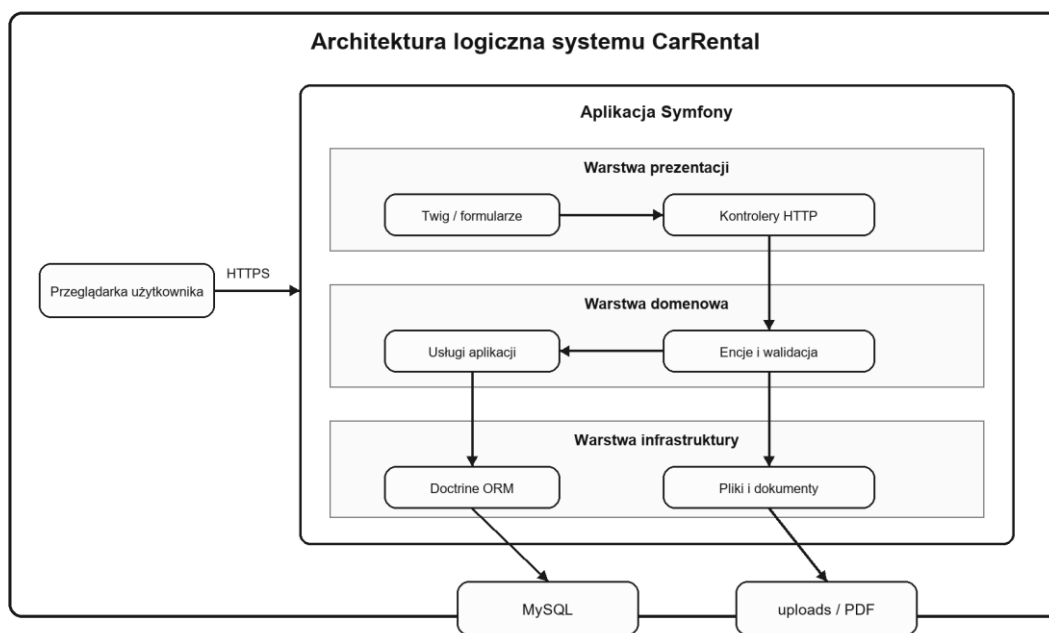


Rysunek 3. Diagram stanów rezerwacji w systemie CarRental.

Wprowadzenie jednoznacznych stanów rezerwacji ogranicza ryzyko niejednoznacznej interpretacji zgłoszeń. Rezerwacja oczekująca wymaga decyzji użytkownika udostępniającego pojazd, rezerwacja zaakceptowana może przejść do stanu aktywnego w czasie trwania wynajmu, a po zakończeniu procesu zostaje zamknięta.

Projekt architektury systemu

Aplikacja została zaprojektowana jako system webowy oparty na architekturze warstwowej. Oddzielenie warstwy prezentacji, logiki domenowej oraz infrastruktury ułatwia rozwój aplikacji i pozwala utrzymywać zależności pomiędzy elementami systemu w uporządkowany sposób.

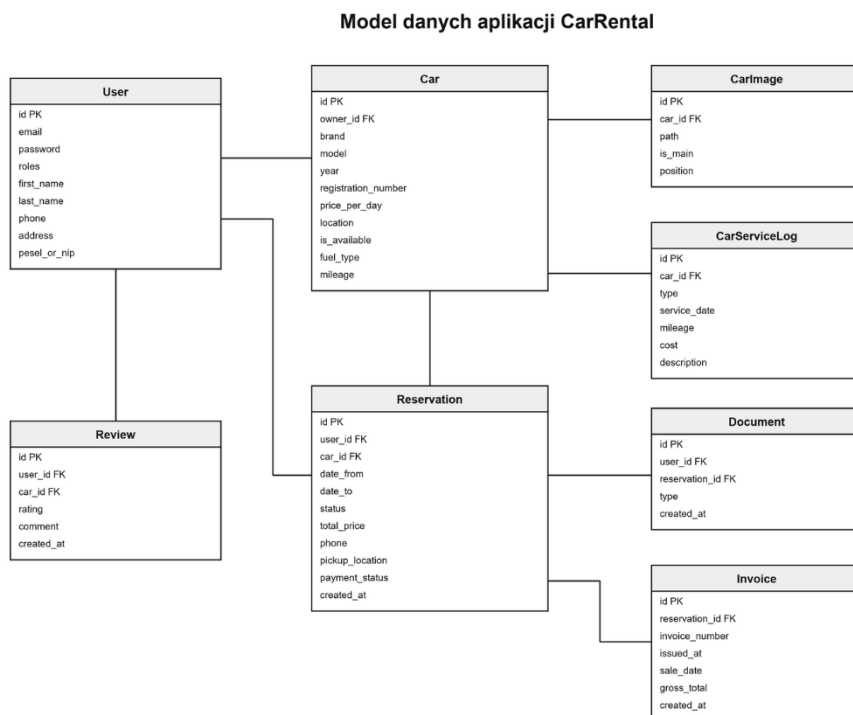


Rysunek 4. Diagram architektury logicznej systemu CarRental.

Warstwa prezentacji odpowiada za obsługę formularzy i widoków Twig. Kontrolery przyjmują żądania HTTP i przekazują dane do usług aplikacyjnych. Encje oraz walidacja reprezentują główne obiekty dziedziny, natomiast Doctrine ORM pośredniczy w komunikacji z bazą MySQL. Pliki przesyłane przez użytkowników oraz dokumenty generowane przez system są przechowywane poza strukturą samej bazy danych.

Projekt modelu danych

Model danych został oparty na encjach odpowiadających najważniejszym obiektom dziedziny: użytkownikom, samochodom, rezerwacjom, opiniom, powiadomieniom, dokumentom oraz logom administracyjnym. Relacje pomiędzy encjami wynikają bezpośrednio z przyjętego procesu wynajmu.



Rysunek 5. Diagram ERD modelu danych aplikacji CarRental.

Centralnym elementem modelu jest rezerwacja łącząca użytkownika z konkretnym samochodem. Samochód jest przypisany do właściciela, a jedna rezerwacja może prowadzić do wygenerowania dokumentu związanego z wynajmem. Powiadomienia oraz logi administracyjne pełnią funkcję pomocniczą, ponieważ dokumentują zdarzenia zachodzące w systemie.

Projekt mechanizmów bezpieczeństwa

Bezpieczeństwo systemu opiera się przede wszystkim na uwierzytelnianiu użytkowników, kontroli ról oraz sprawdzaniu właściciela zasobu przed wykonaniem operacji modyfikujących dane. Użytkownik może zarządzać własnymi rezerwacjami, a operacje dotyczące samochodu są dostępne wyłącznie dla konta, do którego przypisano dany pojazd. Administrator posiada dostęp do funkcji nadzorczych, jednak jego uprawnienia wynikają z roli administracyjnej nadanej w systemie.

W projekcie przewidziano również ochronę formularzy przed atakami CSRF, walidację danych wejściowych oraz przechowywanie haseł w postaci skrótów kryptograficznych. Takie rozwiązania ograniczają ryzyko wykonania nieautoryzowanych operacji oraz zmniejszają skutki ewentualnego błędnego użycia formularzy przez użytkowników.

Podsumowanie projektu systemu

Na etapie projektowym określono sposób działania aplikacji CarRental, zakres odpowiedzialności użytkowników, podstawowe moduły, proces rezerwacji, model danych oraz mechanizmy bezpieczeństwa.

Centralnym elementem systemu jest rezerwacja łącząca użytkownika z wybranym pojazdem i określonym terminem. Wokół tego procesu zbudowano pozostałe funkcje, takie jak zarządzanie samochodami, powiadomienia, opinie oraz dokumentacja.

Przyjęta architektura oparta na Symfony i Doctrine ORM umożliwia oddzielenie warstwy prezentacji, logiki aplikacji i dostępu do danych. Relacyjny model danych pozwala natomiast zachować zależności pomiędzy użytkownikami, pojazdami i rezerwacjami.

Projekt uwzględnia również mechanizmy bezpieczeństwa związane z logowaniem, rolami, kontrolą właściciela zasobu, tokenami CSRF, walidacją formularzy i ochroną danych konfiguracyjnych.

Tak przygotowany projekt stanowi podstawę implementacji poszczególnych elementów aplikacji, której przebieg i wykorzystane rozwiązania techniczne przedstawiono w kolejnym rozdziale.

3. Implementacja systemu CarRental

Organizacja projektu i wykorzystane technologie

Aplikacja CarRental została zaimplementowana z wykorzystaniem frameworka Symfony 6.4 oraz języka PHP. Framework odpowiada za organizację kodu, routing, obsługę formularzy, bezpieczeństwo oraz integrację z bazą danych. Zastosowanie gotowego frameworka pozwoliło uporządkować strukturę aplikacji oraz rozdzielić poszczególne elementy zgodnie z ich odpowiedzialnością.

Warstwa danych wykorzystuje relacyjną bazę MySQL oraz Doctrine ORM. Doctrine umożliwia reprezentowanie rekordów bazy danych za pomocą obiektów PHP, określanych jako encje. Dzięki temu operacje wykonywane na danych mogą być realizowane przy użyciu obiektowego modelu aplikacji, bez konieczności bezpośredniego tworzenia zapytań SQL dla każdej podstawowej operacji.

Warstwa prezentacji została przygotowana z wykorzystaniem systemu szablonów Twig. Pozwala on generować strony HTML na podstawie danych przekazywanych przez kontrolery. Do przygotowania układu strony oraz podstawowych komponentów interfejsu wykorzystano bibliotekę Bootstrap.

Kod aplikacji został podzielony na katalogi odpowiadające poszczególnym warstwom systemu:

- src/Controller – kontrolery obsługujące żądania HTTP;
- src/Entity – encje odwzorowujące strukturę danych;
- src/Form – formularze wykorzystywane podczas wprowadzania danych;
- src/Repository – klasy odpowiedzialne za pobieranie danych;

- src/Security – mechanizmy związane z uwierzytelnianiem;
- src/Service – usługi realizujące wydzieloną logikę;
- templates – szablony Twig odpowiedzialne za interfejs użytkownika;
- public – pliki dostępne bezpośrednio przez serwer WWW, w tym przesłane zdjęcia samochodów;
- config – konfiguracja frameworka i usług aplikacji.

Taki podział pozwala ograniczyć łączenie różnych odpowiedzialności w jednym miejscu i ułatwia późniejsze rozwijanie poszczególnych modułów.

Zastosowane technologie

W tabeli przedstawiono najważniejsze technologie wykorzystane podczas implementacji.

Tabela 4. Technologie wykorzystane podczas implementacji aplikacji

Technologia	Zastosowanie
PHP	Implementacja logiki aplikacji po stronie serwera
Symfony 6.4	Framework aplikacji, routing, formularze i bezpieczeństwo
Doctrine ORM	Mapowanie encji na strukturę relacyjnej bazy danych
MySQL	Przechowywanie danych aplikacji
Twig	Generowanie widoków HTML
Bootstrap	Układ oraz podstawowe komponenty interfejsu
Symfony Security	Logowanie i kontrola dostępu
Symfony Validator	Walidacja danych formularzy
Symfony Mailer/Notifier	Obsługa powiadomień i wiadomości
Dompdf/KnpSnappy	Generowanie dokumentów w formacie PDF
nginx	Serwer WWW w środowisku produkcyjnym
PHP-FPM	Obsługa wykonywania kodu PHP na serwerze
AWS EC2	Środowisko produkcyjne
Git i GitHub	Kontrola wersji kodu źródłowego

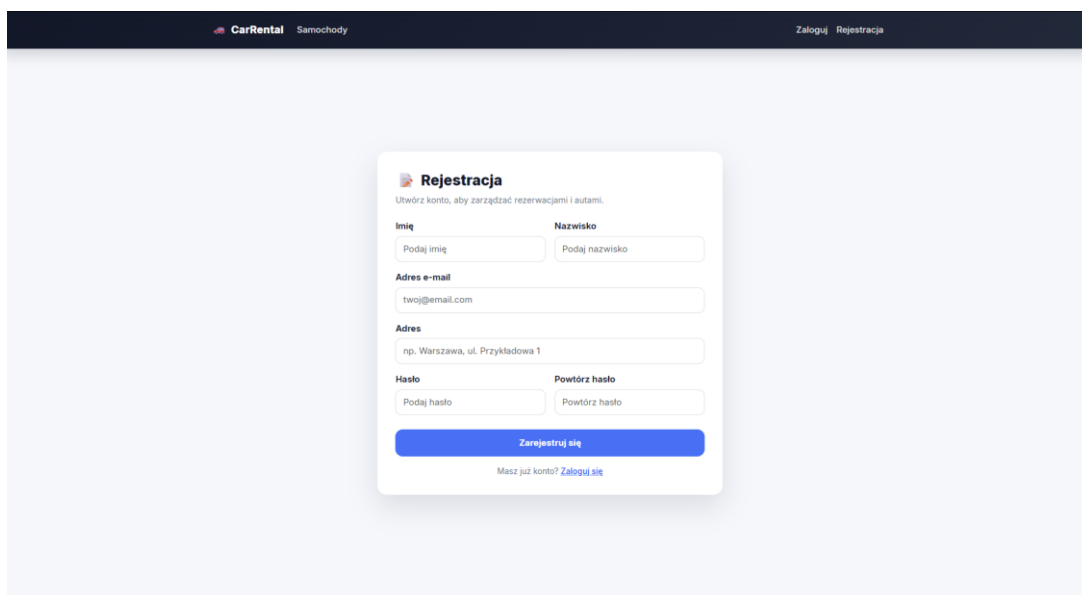
Implementacja obsługi użytkowników

Podstawowym elementem systemu jest mechanizm rejestracji i uwierzytelniania użytkowników. Konto użytkownika reprezentowane jest przez encję `User`, która przechowuje między innymi adres e-mail, rolę, hasło oraz dane profilowe.

Adres e-mail pełni funkcję identyfikatora użytkownika podczas logowania. Hasło nie jest zapisywane w postaci jawnej, lecz przed zapisaniem jest przetwarzane przy wykorzystaniu mechanizmu hashowania dostępnego w Symfony.

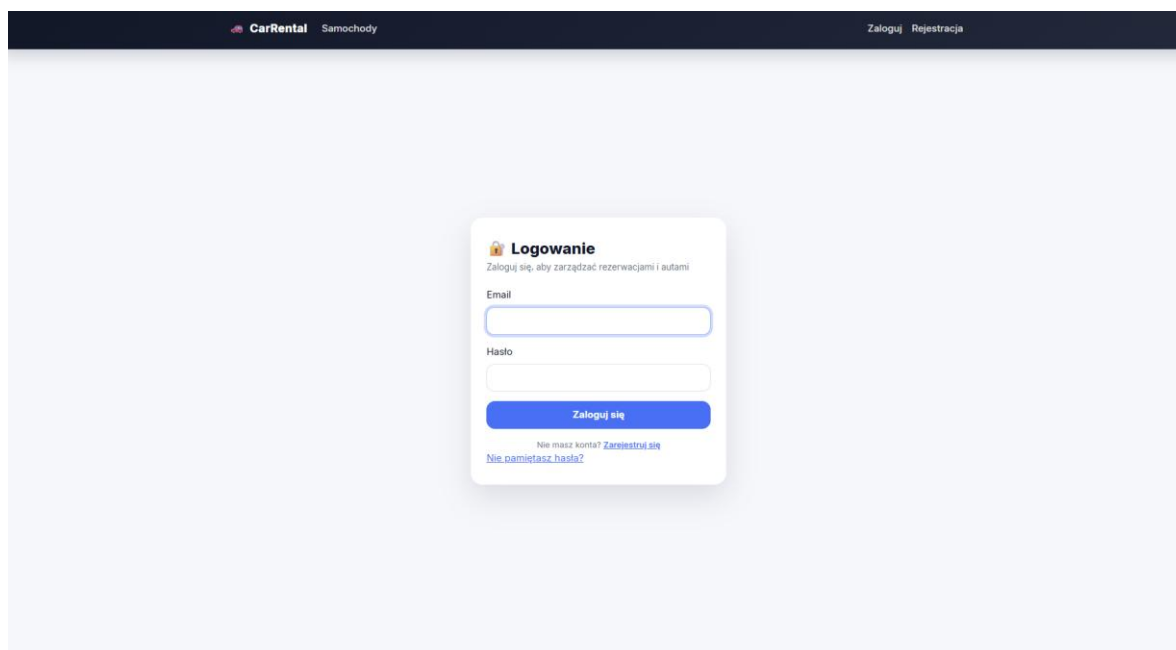
Formularz rejestracji pozwala na wprowadzenie podstawowych danych użytkownika. Oprócz adresu e-mail i hasła system może przechowywać imię, nazwisko, adres, numer

telefonu oraz numer PESEL lub NIP. Dane te są później dostępne w profilu użytkownika oraz mogą być wykorzystywane podczas generowania dokumentów.

The image shows a web application interface for CarRental. At the top, there is a dark blue header with the CarRental logo and the word 'Samochody' on the left, and 'Zaloguj' and 'Rejestracja' on the right. The main content area is light blue. In the center, there is a white registration form titled 'Rejestracja' with a sub-header 'Utwórz konto, aby zarządzać rezerwacjami i autami.' The form contains several input fields: 'Imię' (First Name) with placeholder 'Podaj imię', 'Nazwisko' (Last Name) with placeholder 'Podaj nazwisko', 'Adres e-mail' with placeholder 'twoj@email.com', 'Adres' with placeholder 'np. Warszawa, ul. Przykładowa 1', 'Hasło' (Password) with placeholder 'Podaj hasło', and 'Powtórz hasło' (Repeat Password) with placeholder 'Powtórz hasło'. Below these fields is a blue button labeled 'Zarejestruj się'. At the bottom of the form, there is a link 'Masz już konto? Zaloguj się'.

Rysunek 6. Formularz rejestracji użytkownika w aplikacji CarRental.

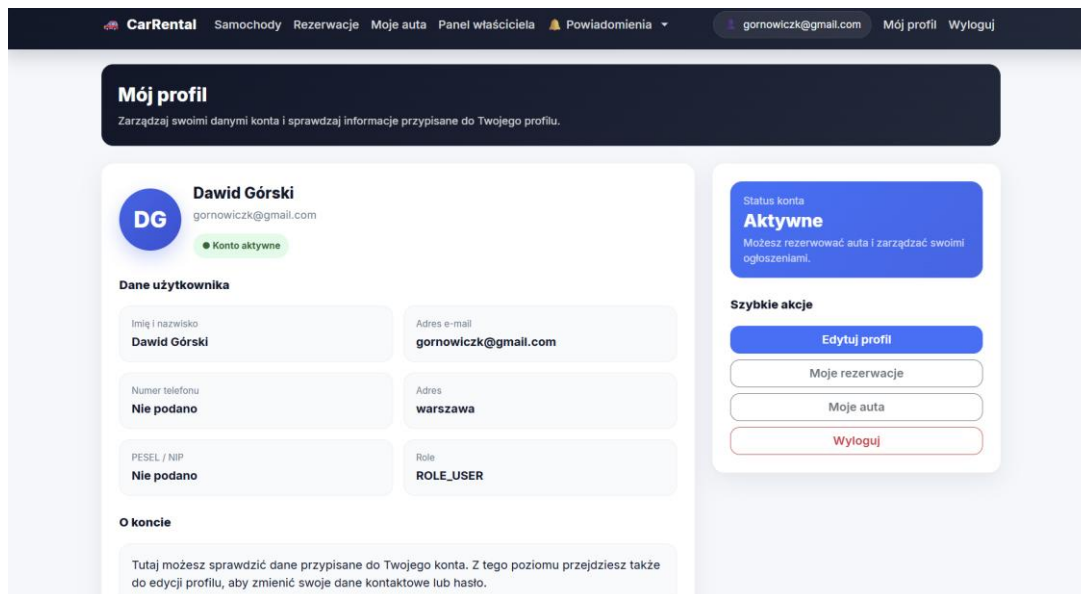
Po poprawnym przesłaniu formularza tworzony jest nowy obiekt `User`, hasło jest hashowane, a następnie dane są zapisywane w bazie przy użyciu `EntityManagerInterface`.

The image shows the same web application interface as before, but with the login form displayed. The header is identical. The main content area is light blue. In the center, there is a white login form titled 'Logowanie' with a sub-header 'Zaloguj się, aby zarządzać rezerwacjami i autami.' The form contains two input fields: 'Email' and 'Hasło'. Below these fields is a blue button labeled 'Zaloguj się'. At the bottom of the form, there are two links: 'Nie masz konta? Zarejestruj się' and 'Nie pamiętasz hasła?'.

Rysunek 7. Ekran logowania użytkownika.

Użytkownik posiada domyślnie rolę `ROLE_USER`. Dodatkowe uprawnienia mogą zostać przyznane poprzez przypisanie kolejnych ról, na przykład `ROLE_ADMIN`.

Po zalogowaniu użytkownik może przejść do własnego profilu. Widok profilu prezentuje podstawowe informacje o koncie i umożliwia przejście do formularza edycji.



Rysunek 8. Widok profilu użytkownika.

Implementacja modułu samochodów

Moduł samochodów został zbudowany wokół encji Car. Każdy pojazd jest przypisany do użytkownika będącego jego właścicielem.

Encja przechowuje między innymi:

- markę;
- model;
- rok produkcji;
- numer rejestracyjny;
- cenę wynajmu za dobę;
- lokalizację;
- informację o dostępności;
- zdjęcie główne;
- dodatkową galerię zdjęć.

Relacja pomiędzy `User` i `Car` pozwala ustalić właściciela danego pojazdu. Jeden użytkownik może posiadać wiele samochodów.

Dodawanie nowego pojazdu odbywa się poprzez formularz `CarType`. Formularz zawiera pola odpowiadające danym encji oraz pola przeznaczone do przesyłania zdjęć.

Dla zdjęć zastosowano walidację typu i rozmiaru pliku. Akceptowane są pliki graficzne w formatach JPEG, PNG oraz WebP, a maksymalny rozmiar pojedynczego pliku wynosi 5 MB.

Po poprawnym przesłaniu formularza zdjęcie otrzymuje wygenerowaną nazwę i jest przenoszone do katalogu przeznaczonego na pliki przesłane przez użytkowników. Nazwa pliku zostaje następnie zapisana w obiekcie samochodu.

Formularz edycji profilu umożliwia aktualizację danych kontaktowych, adresu oraz hasła konta. Widok zachowuje spójny układ z panelem profilu, dzięki czemu użytkownik może łatwo wrócić do swoich rezerwacji lub pojazdów.

CarRental Samochody Rezerwacje Moje auta Panel właściciela Powiadomienia 1 gornowicz1@gmail.com Mój profil Wyloguj

Edytuj profil

Zaktualizuj swoje dane kontaktowe, adres i hasło konta.

Dane użytkownika
Imię: Dawid
Nazwisko: Górski
Numer telefonu: Podaj numer telefonu
Adres zamieszkania: warszawa
Email: gornowicz1@gmail.com
PESEL / NIP: Podaj PESEL lub NIP
Bezpieczeństwo
Jeśli nie chcesz zmieniać hasła, pozostaw pole puste.
Nowe hasło (opcjonalnie): Wpisz nowe hasło jeśli chcesz zmienić.
Zapisz zmiany Powrót do profilu

Edycja konta
Profil
Zadaj o aktualne dane kontaktowe, aby ułatwić komunikację przy rezerwacjach.
Szybkie akcje
Mój profil
Moje rezerwacje
Moje auta
Wyloguj

CarRental — system rezerwacji samochodów Projekt inżynierski

Rysunek 9. Formularz edycji profilu użytkownika.

CarRental Samochody Rezerwacje Moje auta Panel właściciela Powiadomienia 1 gornowicz1@gmail.com Mój profil Wyloguj

Moje auta

Zarządzaj swoimi ogłoszeniami, danymi flotowymi i kontroluj dostępność pojazdów. Dodaj samochód

Nissan Qashqai
2019 • Opole
Niedostępny
Numer rejestracyjny: wd Rok produkcji: 2019
Przebieg: Brak danych VIN: Brak danych
Szczegóły Edytuj Usuń

Jeep Renegade
2018 • Zakopane
Dostępny
Numer rejestracyjny: fe Rok produkcji: 2018
Przebieg: Brak danych VIN: Brak danych
Szczegóły Edytuj Usuń

Tesla Model 3
2021 • Kraków
Dostępny
Numer rejestracyjny: dew Rok produkcji: 2021
Przebieg: Brak danych VIN: Brak danych
Szczegóły Edytuj Usuń

Volvo V60
2020 • Warszawa
Dostępny
Numer rejestracyjny: ceaw Rok produkcji: 2020
Przebieg: Brak danych VIN: Brak danych
Szczegóły Edytuj Usuń

Fiat Tipo
2019 • Olsztyn
Dostępny
Numer rejestracyjny: few Rok produkcji: 2019
Przebieg: Brak danych VIN: Brak danych
Szczegóły Edytuj Usuń

Seat Leon
2021 • Gdynia
Dostępny
Numer rejestracyjny: dqw Rok produkcji: 2021
Przebieg: Brak danych VIN: Brak danych
Szczegóły Edytuj Usuń

Rysunek 10. Widok listy samochodów dodanych przez użytkownika.

Rysunek 11. Formularz dodawania samochodu - podstawowe dane pojazdu.

Rysunek 12. Formularz dodawania samochodu - dane techniczne, zdjęcia i zapis ogłoszenia.

Prezentacja ofert i filtrowanie samochodów

Lista samochodów prezentuje oferty w formie kart zawierających zdjęcie pojazdu, markę, model, cenę wynajmu za dobę, rok produkcji oraz status dostępności. Po lewej stronie widoku znajduje się panel filtrów pozwalający zawęzić wyniki wyszukiwania.

Mechanizm filtrowania pozwala ograniczyć listę samochodów do pojazdów spełniających wybrane kryteria. Dostępne kryteria obejmują:

- markę;
- model;
- minimalny i maksymalny rok produkcji;

- minimalną i maksymalną cenę;
- lokalizację;
- dostępność.

Repozytorium buduje zapytanie przy wykorzystaniu QueryBuilder. Poszczególne warunki są dodawane tylko wtedy, gdy użytkownik poda odpowiednią wartość filtra, dzięki czemu jeden mechanizm obsługuje zarówno pełną listę pojazdów, jak i zawężone wyniki wyszukiwania.

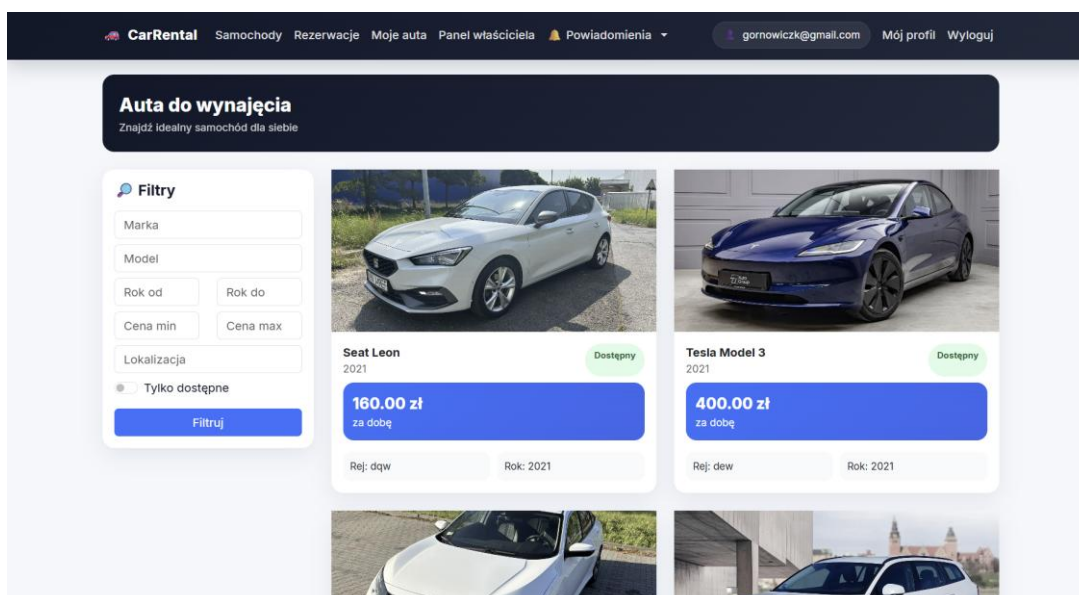
Przykładowo filtr dotyczący minimalnej ceny dodaje warunek porównujący wartość pola pricePerDay z wartością podaną przez użytkownika. Analogicznie działają pozostałe kryteria, co pozwala ograniczyć liczbę wyświetlanych ofert bez zmiany struktury widoku.

Po dodaniu pojazdu użytkownik może ponownie otworzyć formularz edycji samochodu i zaktualizować dane oferty. Ekran obejmuje podstawowe informacje, parametry techniczne oraz ustawienia dostępności.

Rysunek 13. Formularz edycji samochodu - podstawowe dane i dostępność.

Dolna część formularza edycji samochodu pozwala zarządzać aktualnymi zdjęciami pojazdu oraz dodać nowe pliki graficzne. W tym widoku właściciel może usunąć zdjęcie główne, zmienić galerię i zapisać zaktualizowane ogłoszenie.

Rysunek 14. Formularz edycji samochodu - zarządzanie zdjęciami pojazdu.



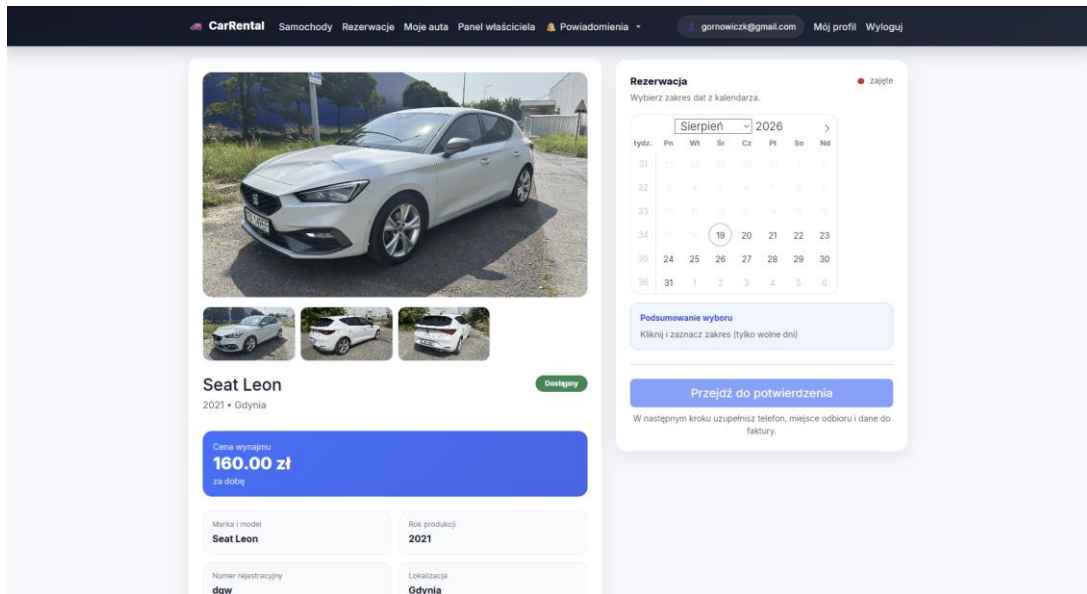
Rysunek 15. Polecane auta prezentowane na stronie głównej aplikacji.

Widok szczegółów pojazdu

Po wybraniu samochodu użytkownik przechodzi do strony szczegółów. Widok prezentuje podstawowe dane pojazdu, cenę wynajmu, lokalizację, dostępność oraz zdjęcia.

Jeżeli samochód posiada zdjęcie główne, jest ono prezentowane jako pierwszy element galerii. W przypadku braku zdjęcia system może wyświetlić komunikat informujący o braku fotografii.

Widok szczegółów zawiera również sekcję opinii oraz informacje o dostępności pojazdu.



Rysunek 16. Widok szczegółów samochodu z kalendarzem rezerwacji.

Implementacja procesu rezerwacji

Proces rezerwacji stanowi jeden z najważniejszych elementów aplikacji. Dane rezerwacji reprezentowane są przez encję Reservation.

Encja przechowuje powiązanie z użytkownikiem i samochodem oraz datę rozpoczęcia, datę zakończenia, status i dodatkowe dane związane z procesem.

Podczas tworzenia rezerwacji użytkownik wybiera przedział czasu. Dane mogą zostać pobrane z formularza i następnie wykorzystane do utworzenia nowego obiektu Reservation.

Przed zapisaniem rezerwacji aplikacja powinna sprawdzić, czy termin nie koliduje z istniejącymi rezerwacjami dotyczącymi tego samego samochodu.

Jeżeli termin jest dostępny, tworzony jest nowy obiekt rezerwacji, do którego przypisywany jest aktualnie zalogowany użytkownik, wybrany pojazd oraz daty wynajmu. Status początkowy przyjmuje wartość odpowiadającą rezerwacji oczekującej.

Przykładowy fragment kodu odpowiedzialnego za utworzenie rezerwacji:

```
$reservation = new Reservation();  
$reservation->setCar($car);  
$reservation->setUser($user);  
$reservation->setStartDate($start);  
$reservation->setEndDate($end);
```

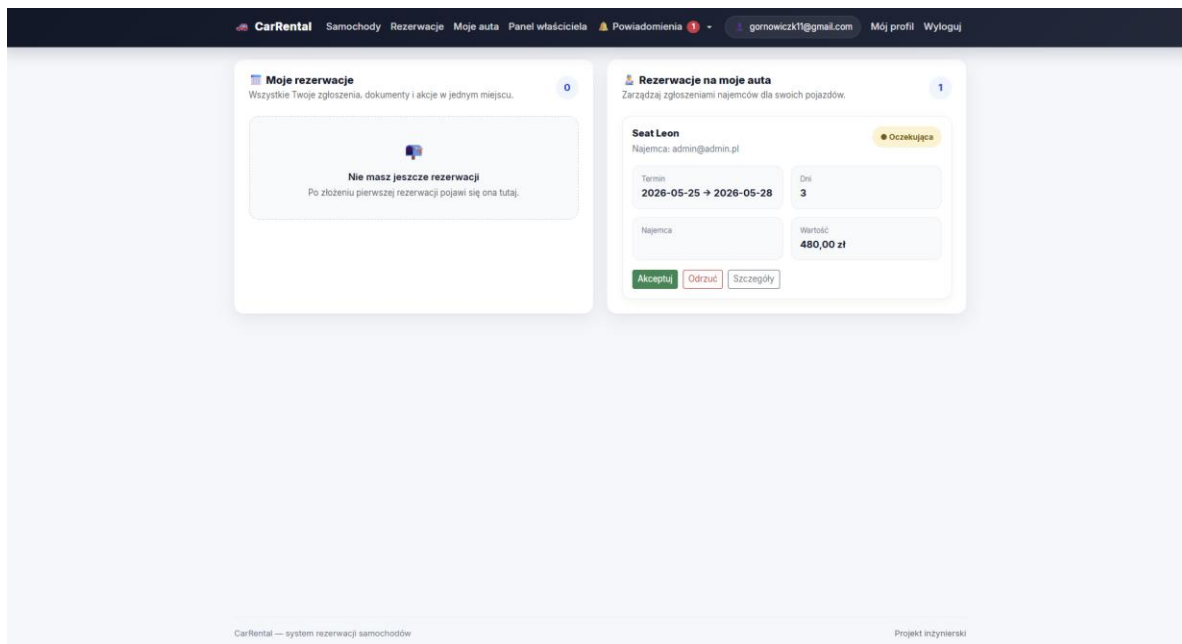
```
$reservation->setStatus('pending');
```

```
$this->em->persist($reservation);
```

```
$this->em->flush();
```

Listing 1. Przykład tworzenia obiektu rezerwacji

Utworzenie rezerwacji nie oznacza jeszcze automatycznej zgody właściciela. Właściciel może przejść do widoku zgłoszeń dotyczących własnych pojazdów i podjąć decyzję o zaakceptowaniu lub odrzuceniu zgłoszenia.



Rysunek 17. Lista rezerwacji obsługiwanych przez użytkownika udostępniającego pojazd.

Potwierdzenie rezerwacji

Przed ostatecznym zapisaniem rezerwacji system może zaprezentować użytkownikowi podsumowanie zawierające wybrany samochód, termin oraz wyliczony koszt.

Rysunek 18. Potwierdzenie rezerwacji wraz z podsumowaniem kosztów.

Koszt może zostać obliczony na podstawie liczby dni oraz wartości pricePerDay przechowywanej dla samochodu.

Rysunek 19. Widok rezerwacji przypisanych do zalogowanego użytkownika.

Obsługa statusów

W systemie stosowane są statusy pozwalające określić aktualny stan rezerwacji. W podstawowym zakresie występują między innymi:

- pending – rezerwacja oczekująca;
- accepted – rezerwacja zaakceptowana;
- rejected – rezerwacja odrzucona;

- completed – rezerwacja zakończona.

Właściciel pojazdu może zaakceptować lub odrzucić rezerwację. Zmiana powoduje aktualizację wartości pola status w bazie danych.

Na stronie rezerwacji użytkownik widzi aktualny status w formie odpowiednio oznaczonej etykiety.

Właściciel posiada oddzielny widok zgłoszeń dotyczących jego samochodów. Dla oczekujących rezerwacji dostępne są operacje zaakceptowania lub odrzucenia.

Implementacja mechanizmu dostępności

Informacje o dostępności samochodu są powiązane z jego rezerwacjami.

Dla określonego samochodu pobierane są rezerwacje posiadające odpowiednie statusy. Na ich podstawie tworzona jest lista zajętych przedziałów czasu.

Kontroler może zwrócić dane w formacie JSON, które następnie są wykorzystywane w widoku szczegółów pojazdu.

Przykładowa struktura pojedynczego wpisu zawiera:

```
[
  'title' => 'Zarezerwowane',
  'start' => $r->getStartDate()->format('Y-m-d'),
  'end' => (clone $r->getEndDate())
  ->modify('+1 day')
  ->format('Y-m-d'),
]
```

Listing 2. Przygotowanie danych dotyczących dostępności pojazdu

Dzięki temu użytkownik przed utworzeniem rezerwacji może sprawdzić, które terminy są już zajęte.

Implementacja opinii użytkowników

Opinie o samochodach przechowywane są przy użyciu encji Review.

Każda opinia jest powiązana z samochodem i użytkownikiem. Zawiera ocenę liczbową w zakresie od 1 do 5 oraz treść komentarza.

Formularz ReviewType pozwala wybrać ocenę i wprowadzić tekst opinii. Dane są walidowane przed zapisaniem.

Na stronie szczegółów samochodu wyświetlana jest lista opinii oraz średnia ocena obliczana na podstawie zapisanych wartości.

Szczegółowy widok rezerwacji gromadzi informacje o pojeździe, terminie, najemcy, właścicielu oraz aktualnym statusie zgłoszenia. Z tego poziomu osoba udostępniająca auto lub administrator może zaakceptować albo odrzucić rezerwację.

Szczegóły rezerwacji Oczekująca

Seat Leon • 2026-05-25 → 2026-05-28

Informacje o rezerwacji

Samochód: Seat Leon | Numer rejestracyjny: dqw

Rok: 2021 | Okres rezerwacji: 2026-05-25 → 2026-05-28

Liczba dni: 3 | Cena za dobę: 160,00 zł

Najemca | **Właściciel**

admin@admin.pl | Telefon: 24525252 | Miejsce odbioru: warszawa

Dawid Gorski | gornowicz11@gmail.com

Dokumenty

Dokumenty będą dostępne po akceptacji rezerwacji.

Zmiana terminu

Szybkie informacje

Status: Oczekująca | Auto: Seat Leon

Telefon: 24525252 | Odbiór: warszawa

Akcje

Szczegóły pojazdu

Akceptuj rezerwację

Odrzuć rezerwację

← Powrót

Łączna kwota: 480,00 zł
3 dni • 160,00 zł / doba

Rysunek 20. Widok szczegółów rezerwacji z aktualnym statusem i dostępnymi akcjami.

Rezerwacja zajęte

Wybierz zakres dat z kalendarza.

Sierpień 2026

tydz. Pn Wt Śr Cz Pi So Nd

31 27 28 29 30 31 1 2

32 3 4 5 6 7 8 9

33 10 11 12 13 14 15 16

34 17 18 19 20 21 22 23

35 24 25 26 27 28 29 30

36 31 1 2 3 4 5 6

Podsumowanie wyboru

Kliknij i zaznacz zakres (tylko wolne dni)

Przejdź do potwierdzenia

W następnym kroku uzupełnij telefon, miejsce odbioru i dane do faktury.

Opinie 0.0/5 ★ ★ ★ ★ ★ (0 opinii)

Brak opinii dla tego auta.

Dodaj opinię

Ocena: ★ ★ ★ ★ ★ (5)

Podziel się wrażeniami z wynajmu...

Treść opinii

Wyślij opinię

Rysunek 21. Sekcja opinii użytkowników na stronie pojazdu.

Użytkownik może również usunąć własną opinię. Operacja ta powinna być ograniczona do autora wpisu.

Implementacja powiadomień

Powiadomienia służą do przekazywania użytkownikom informacji dotyczących zdarzeń zachodzących w aplikacji.

Encja `Notification` przechowuje użytkownika, treść komunikatu, informację o odczytaniu oraz datę utworzenia.

Powiadomienia przypisane do aktualnie zalogowanego użytkownika mogą zostać pobrane z bazy i wyświetlone na odpowiedniej stronie.

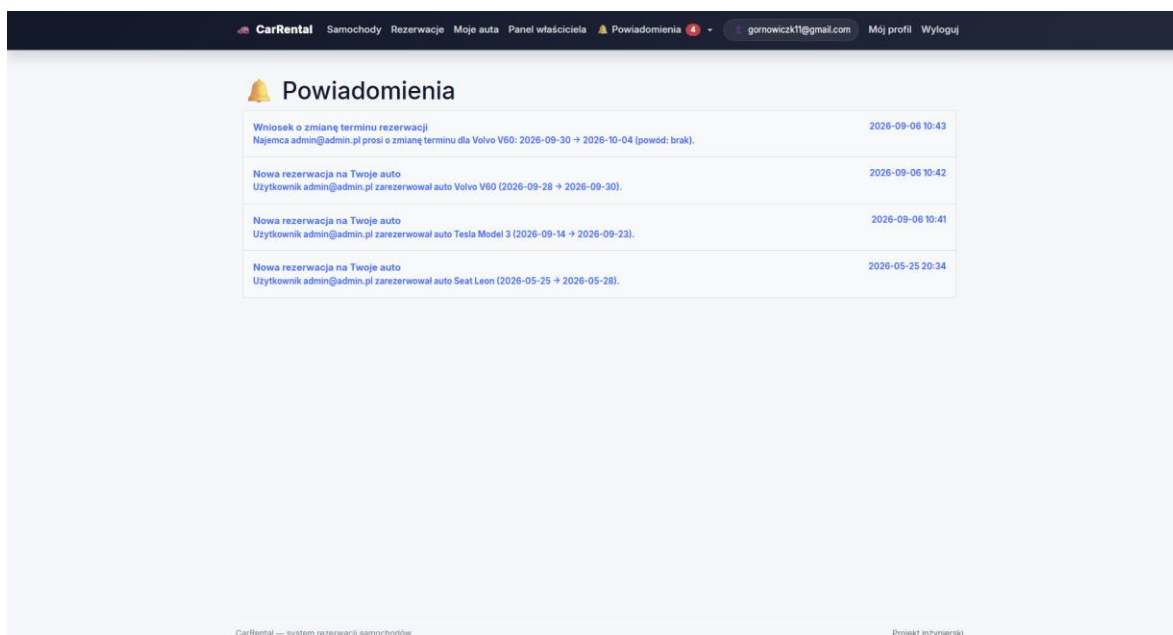
Użytkownik może oznaczyć powiadomienie jako przeczytane. Przed wykonaniem operacji sprawdzane jest, czy dane powiadomienie rzeczywiście należy do aktualnego użytkownika.

Przykładowa kontrola:

```
if ($notification->getUser() !== $this->getUser()) {  
    throw $this->createAccessDeniedException();  
}
```

Listing 3. Weryfikacja właściciela powiadomienia

Operacja oznaczenia powiadomienia jako przeczytane jest dodatkowo chroniona tokenem CSRF.



Rysunek 22. Lista powiadomień użytkownika.

Generowanie dokumentów związanych z wynajmem

System przewiduje możliwość generowania dokumentów związanych z rezerwacją.

Dane potrzebne do przygotowania dokumentu mogą być pobierane z już istniejących obiektów, takich jak użytkownik, samochód i rezerwacja. Pozwala to ograniczyć konieczność ponownego ręcznego wprowadzania informacji.

Na podstawie rezerwacji można wykorzystać między innymi:

- dane użytkownika;
- dane samochodu;
- numer rejestracyjny;
- datę rozpoczęcia;
- datę zakończenia;
- cenę wynajmu.

Dokumenty mogą być następnie generowane w formacie PDF.

W systemie przewidziano między innymi dokument związany z umową wynajmu oraz dokument rozliczeniowy.

FAKTURA VAT

Rezerwacja: #4

Okres najmu:

Numer: FV/09/2026/0001

2026-09-14 → 2026-09-23

Data wystawienia: 2026-09-06

Data sprzedaży: 2026-09-14

Sprzedawca CarRental Sp. z o.o. ul. Testowa 1, 00-000 Warszawa NIP: 123-456-78-90 Bank: mBank IBAN: PL00123456789012345678901234	Nabywca admin@admin.pl Kwiatowa 14, 75-235 warszawa NIP: 42424252
---	--

Lp.	Nazwa usługi	Ilość	Cena brutto / doba	Netto	VAT	Brutto
1	Wynajem samochodu Tesla Model 3 (dew) Okres: 2026-09-14 → 2026-09-23	9 dób	400,00 zł	2 926,83 zł	673,17 zł	3 600,00 zł

Razem netto	2 926,83 zł
Razem VAT (23%)	673,17 zł
Do zapłaty brutto	3 600,00 zł

Informacje o płatności

Sposób płatności: przelew / płatność online

Numer rachunku: PL00123456789012345678901234

Tytuł płatności: FV/09/2026/0001

Dokument wygenerowany automatycznie przez system CarRental.

Rysunek 23. Przykład faktury VAT wygenerowanej na podstawie danych rezerwacji.

Generowanie numeru faktury

W projekcie przewidziano mechanizm umożliwiający nadawanie kolejnym dokumentom rozliczeniowym uporządkowanych numerów.

Numer może zawierać rok, miesiąc oraz kolejny numer dokumentu w danym okresie.

Przykładowy format:

FV/2026/08/0001

Mechanizm może wyszukać największy istniejący numer dla danego miesiąca i na jego podstawie wyznaczyć następny numer.

Przykład:

```
$year = $baseDate->format('Y');  
$month = $baseDate->format('m');  
$prefix = sprintf('FV/%s/%s/', $year, $month);  
  
$next = 1;
```

```
return $prefix . str_pad(  
    (string) $next,  
    4,  
    '0',  
    STR_PAD_LEFT  
);
```

Listing 4. Fragment mechanizmu generowania numeru dokumentu rozliczeniowego

Takie rozwiązanie zapewnia czytelny i uporządkowany sposób identyfikacji dokumentów generowanych przez aplikację.

Panel administratora

Panel administratora jest dostępny wyłącznie dla użytkowników posiadających rolę `ROLE_ADMIN`.

Ograniczenie dostępu zostało zrealizowane przy użyciu atrybutu:

```
# [IsGranted('ROLE_ADMIN')]
```

Umieszczenie go na poziomie kontrolera administracyjnego powoduje ograniczenie dostępu do wszystkich tras zdefiniowanych w tej klasie.

Panel administratora może prezentować między innymi informacje dotyczące liczby użytkowników, samochodów i rezerwacji.

Administrator może przejść do widoków służących do zarządzania poszczególnymi zasobami.

Drugim przykładem dokumentu generowanego w aplikacji jest protokół wydania i zwrotu pojazdu. Dokument porządkuje informacje potrzebne przy przekazaniu

samochodu, takie jak dane pojazdu, strony najmu, stan licznika, poziom paliwa lub naładowania oraz miejsce na podpisy.

PROTOKÓŁ WYDANIA I ZWROTU POJAZDU

Rezerwacja #4 • dokument wygenerowany 2026-09-06 10:44

1. DANE POJAZDU

Pojazd: Tesla Model 3
Rok produkcji: 2021
Numer rejestracyjny: dew
VIN: —
Okres najmu: 2026-09-14 → 2026-09-23
Miejsce odbioru / zwrotu: warszawa

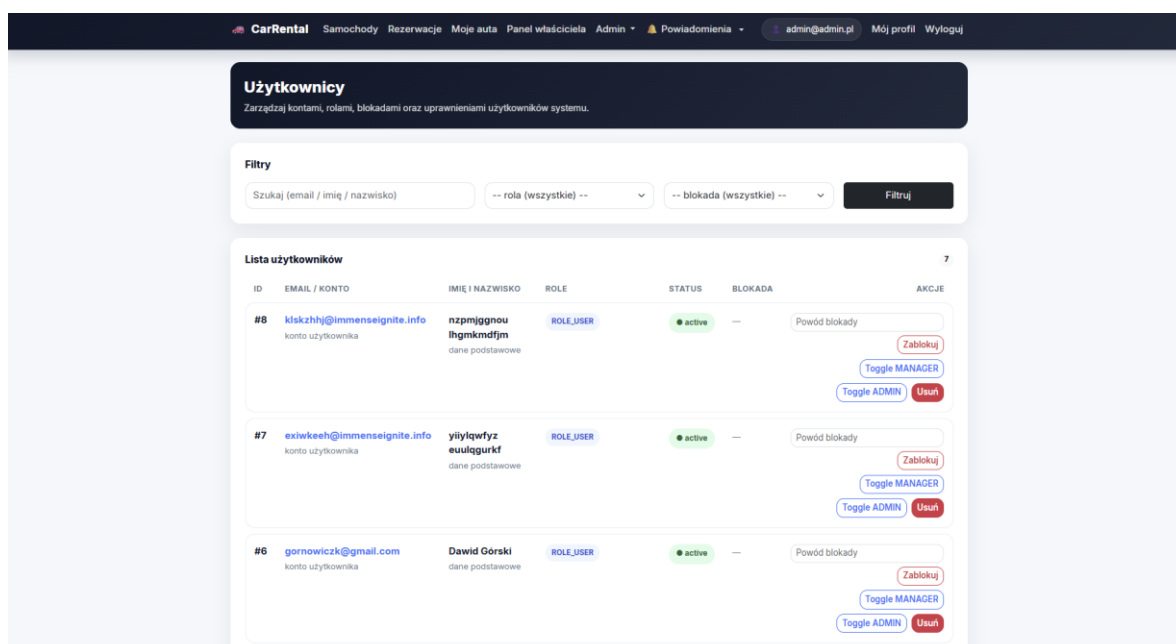
2. STRONY

Najemca	Wynajmujący
admin@admin.pl Email: admin@admin.pl Telefon: 6437637263	Dawid Górski Email: gornowicz11@gmail.com Telefon: —

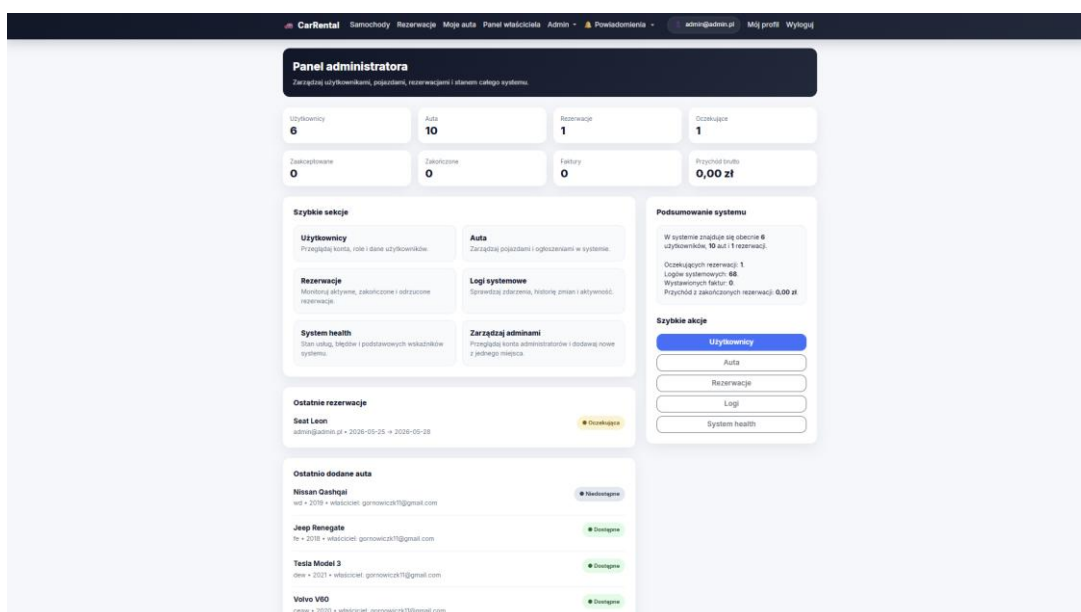
3. WYDANIE POJAZDU

Data i godzina wydania	
Przebieg przy wydaniu	 km
Poziom paliwa / naładowania	
Liczba kluczyków	
Dokumenty pojazdu	TAK / NIE
Stan pojazdu przy wydaniu	

Rysunek 24. Przykład protokołu wydania i zwrotu pojazdu wygenerowanego przez system.

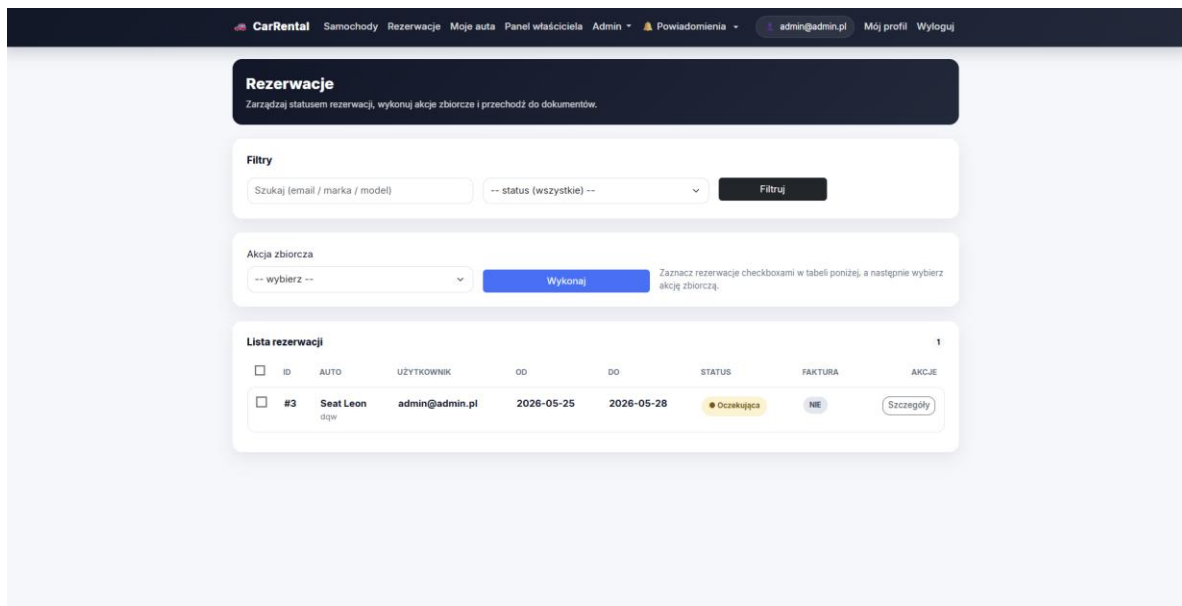


Rysunek 25. Panel administratora - zarządzanie użytkownikami, rolami i blokadami kont.



Rysunek 26. Główny panel administratora systemu CarRental.

W kontrolerze znajdują się również operacje dotyczące usuwania użytkowników i samochodów oraz obsługi rezerwacji.



Rysunek 27. Panel administratora - lista rezerwacji z filtrowaniem i statusem.

Operacje zmieniające dane wykonywane są metodą POST i mogą wymagać prawidłowego tokenu CSRF.

Przykład:

```
if (!$this->isCsrfTokenValid(
    'admin_delete_car_'.$car->getId(),
    $request->request->get('_token')
)) {
    throw $this->createAccessDeniedException(
        'Błędny token.'
    );
}
```

Listing 5. Weryfikacja tokenu CSRF podczas operacji administracyjnej

Implementacja mechanizmów bezpieczeństwa

Oprócz kontroli ról aplikacja stosuje dodatkowe mechanizmy zabezpieczające operacje wykonywane przez użytkowników.

Kontrola właściciela zasobu

W przypadku usuwania samochodu sprawdzane jest, czy pojazd należy do aktualnie zalogowanego użytkownika.

```
if ($car->getOwner() !== $this->getUser()) {
    throw $this->createAccessDeniedException();
}
```

}

Listing 6. Kontrola właściciela samochodu przed wykonaniem operacji

Dzięki temu użytkownik nie może usunąć samochodu należącego do innej osoby poprzez samą zmianę identyfikatora w adresie żądania.

Ochrona przed CSRF

Operacje zmieniające dane, takie jak usunięcie samochodu, akceptacja rezerwacji lub oznaczenie powiadomienia jako przeczytane, mogą wymagać tokenu CSRF.

Token jest generowany w widoku, a następnie weryfikowany w kontrolerze.

Walidacja danych

Formularze Symfony pozwalają na określenie ograniczeń walidacyjnych.

Dla przykładu pole ceny za dobę musi zawierać wartość dodatnią, rok produkcji jest ograniczony do określonego zakresu, a przesyłane zdjęcia muszą posiadać odpowiedni typ MIME.

Mechanizmy te pozwalają ograniczyć zapisywanie nieprawidłowych danych w bazie.

Interfejs użytkownika

Warstwa prezentacji została przygotowana przy użyciu szablonów Twig oraz komponentów Bootstrap.

Główny szablon base.html.twig zawiera wspólny układ aplikacji, w tym pasek nawigacyjny, główną część strony oraz stopkę. Dzięki temu poszczególne widoki zachowują spójny układ i nie powielają tych samych elementów interfejsu.

Elementy menu zależą od stanu uwierzytelnienia użytkownika. Osoba niezalogowana otrzymuje dostęp do logowania i rejestracji, natomiast zalogowany użytkownik może przejść do swoich rezerwacji, samochodów, profilu oraz funkcji związanych z obsługą wynajmu.

 **CarRental**



Auta do wynajęcia

Znajdź idealny samochód dla siebie

 **Filtry**

Marka

Model

Rok od

Rok do

Cena min

Cena max

Lokalizacja

☐ Tylko dostępne

Filtruj

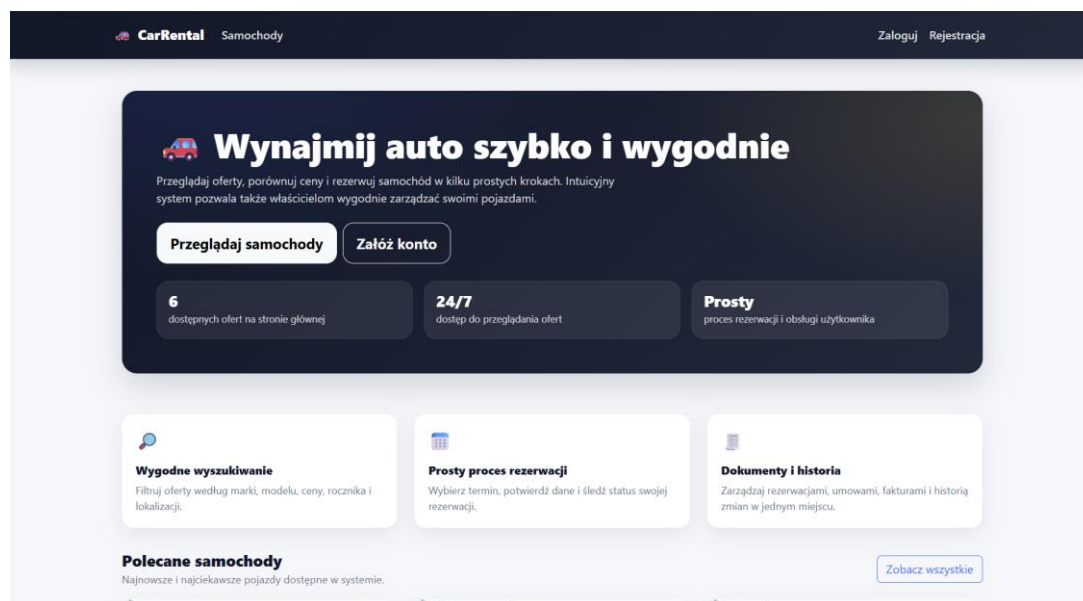


Rysunek 28. Przykład responsywnego widoku aplikacji na ekranie o mniejszej szerokości.

Wspólne komunikaty systemowe zostały wydzielone do częściowego szablonu, dzięki czemu mogą być wykorzystywane w wielu widokach bez powielania kodu i bez zmiany układu każdej strony osobno.

W interfejsie wykorzystano karty ofert, formularze, tabele oraz oznaczenia statusów dostępne w Bootstrapie. Strona główna pełni rolę punktu wejścia do aplikacji:

prezentuje podstawową nawigację, wyszukiwarkę oraz wybrane samochody, z których użytkownik może przejść do dalszych operacji.



Rysunek 29. Strona główna aplikacji CarRental.

Wdrożenie aplikacji

Wdrożenie aplikacji można przedstawić jako powtarzalną procedurę uruchomienia systemu w środowisku produkcyjnym. W projekcie wykorzystano instancję AWS EC2 z systemem Ubuntu Server, serwer nginx jako warstwę dostępu, PHP-FPM do obsługi kodu PHP oraz bazę danych MySQL.

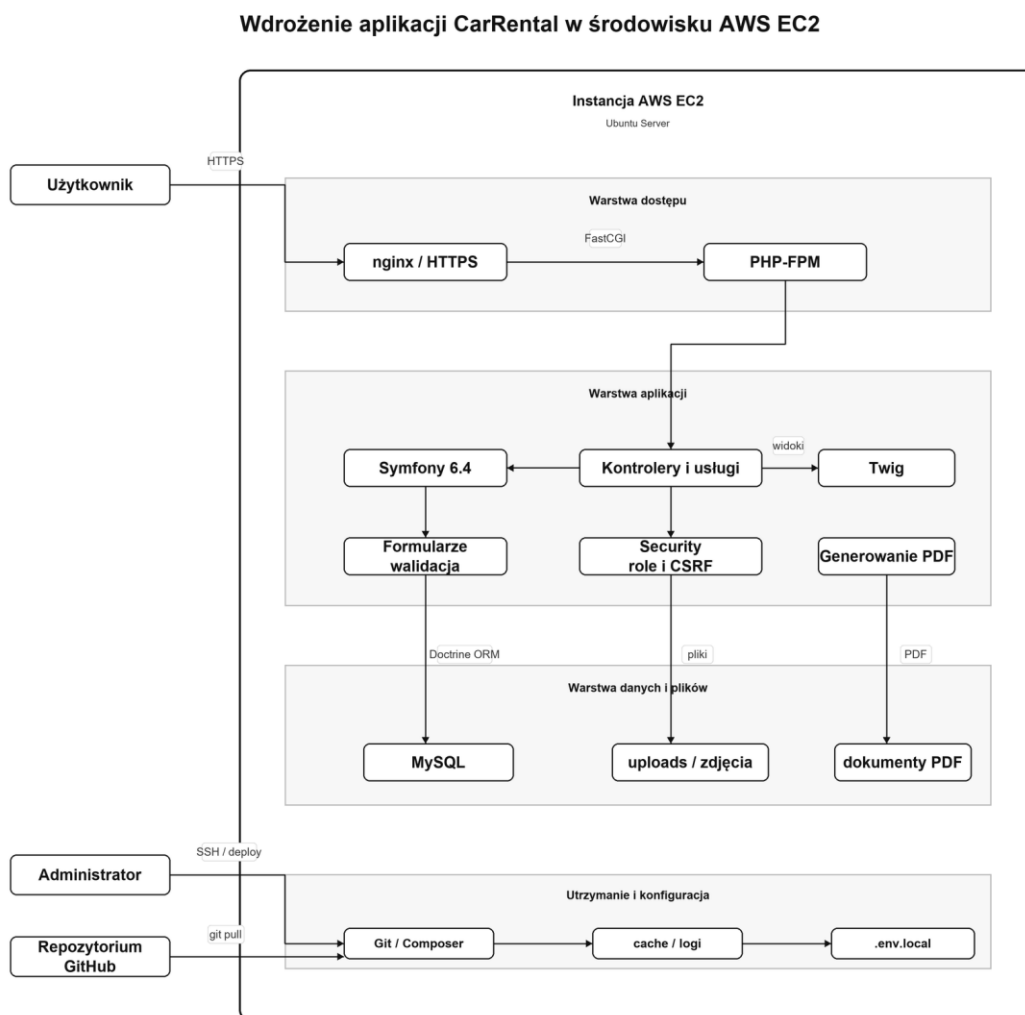
Kolejność czynności wdrożeniowych rozpoczyna się od przygotowania instancji serwerowej: aktualizacji pakietów, instalacji PHP z wymaganymi rozszerzeniami, instalacji Composer, konfiguracji nginx oraz przygotowania katalogu aplikacji w lokalizacji `/var/www/rental-app`.

Następnie na serwer pobierany jest kod źródłowy aplikacji z repozytorium GitHub. Po pobraniu kodu instalowane są zależności Composer, a prawa dostępu do katalogów cache, logów, plików przesyłanych przez użytkowników i dokumentów generowanych przez system są ustawiane tak, aby mogły być zapisywane przez proces aplikacji.

Kolejnym etapem jest przygotowanie konfiguracji produkcyjnej. Dane dostępne do bazy MySQL, klucze aplikacji i inne wartości wrażliwe są przechowywane w pliku konfiguracyjnym znajdującym się na serwerze, a nie w repozytorium kodu. Takie rozwiązanie ogranicza ryzyko przypadkowego opublikowania haseł lub sekretów.

Po skonfigurowaniu środowiska wykonywane są czynności aplikacyjne: przygotowanie lub aktualizacja schematu bazy danych, wyczyszczenie cache dla środowiska produkcyjnego oraz restart usług nginx i PHP-FPM. Ostatnim krokiem jest kontrola

działania aplikacji w przeglądarce, obejmująca logowanie, wyświetlanie ofert, składanie rezerwacji, obsługę zdjęć oraz generowanie dokumentów PDF.



Rysunek 30. Diagram wdrożenia aplikacji w środowisku AWS EC2.

Diagram wdrożenia przedstawia zależności między użytkownikiem, administratorem, instancją AWS EC2 oraz głównymi warstwami aplikacji. Ruch użytkownika trafia przez HTTPS do nginx, następnie do PHP-FPM i warstwy aplikacji Symfony. Administrator korzysta z połączenia SSH podczas aktualizacji kodu, instalacji zależności, czyszczenia cache oraz utrzymania konfiguracji produkcyjnej.

Problemy napotkane podczas implementacji i wdrożenia

Podczas realizacji projektu wystąpiły problemy typowe dla aplikacji webowych rozwijanych równocześnie w środowisku lokalnym i produkcyjnym.

Jednym z problemów były błędne ścieżki do plików przesyłanych przez użytkowników. Zdjęcie mogło zostać poprawnie zapisane na serwerze, lecz nie było dostępne pod adresem generowanym przez aplikację. Rozwiązanie wymagało ujednolicenia

lokalizacji katalogu uploadów oraz sposobu tworzenia adresów do plików w szablonach Twig.

Kolejny problem dotyczył konfiguracji bazy danych. Niepoprawne dane dostępne do MySQL uniemożliwiały aplikacji nawiązanie połączenia. Wymagało to zweryfikowania konfiguracji użytkownika bazy oraz zmiennych środowiskowych.

Podczas wdrożenia konieczne było również zwrócenie uwagi na środowisko produkcyjne Symfony. Tryb debugowania powinien być wyłączony, ponieważ komunikaty błędów mogą ujawniać informacje techniczne dotyczące struktury aplikacji.

W czasie rozwijania projektu pojawiały się również problemy związane z nazwami tras używanymi w szablonach. Odwołanie do nieistniejącej lub wcześniej zmienionej trasy powodowało błąd generowania adresu URL. Rozwiązaniem było ujednolicenie nazw tras pomiędzy kontrolerami a szablonami.

Istotnym problemem bezpieczeństwa było również właściwe zarządzanie plikami konfiguracyjnymi. Dane zawierające hasła i sekrety nie powinny być publikowane w repozytorium Git. Konfigurację produkcyjną przeniesiono do plików środowiskowych znajdujących się bezpośrednio na serwerze

Wykrywanie i rozwiązywanie wymienionych problemów odbywało się przy wykorzystaniu komunikatów frameworka Symfony, logów aplikacji, logów serwera oraz poleceń wykonywanych w terminalu.

Najważniejsze problemy napotkane podczas implementacji i wdrożenia zestawiono w tabeli 5.

Tabela 5. Zestawienie problemów napotkanych podczas implementacji i wdrożenia

Problem	Przyczyna	Zastosowane rozwiązanie
Zdjęcia samochodów nie były dostępne	Niepoprawna ścieżka do katalogu przesłanych plików	Ujednolicono katalog uploadów i generowanie adresów
Brak połączenia z bazą MySQL	Niepoprawne dane konfiguracyjne	Poprawiono konfigurację środowiskową
Błąd generowania adresu strony	Niepoprawna lub zmieniona nazwa trasy	Ujednolicono nazwy tras w kontrolerach i szablonach
Tryb debug w środowisku produkcyjnym	Błędna konfiguracja środowiska	Wyłączono tryb debug
Brak konfiguracji po usunięciu sekretów z repozytorium	Dane środowiskowe nie znajdowały się na serwerze	Utworzono lokalną konfigurację produkcyjną
Ryzyko kolizji rezerwacji	Brak odpowiedniej kontroli terminów	Dodano sprawdzanie dostępności pojazdu

Podsumowanie implementacji

W ramach implementacji przygotowano najważniejsze moduły wymagane do obsługi procesu wynajmu pojazdu pomiędzy użytkownikami.

Zaimplementowano mechanizmy rejestracji i logowania, profile użytkowników, zarządzanie samochodami, przysyłanie zdjęć, filtrowanie ofert, rezerwacje, statusy, dostępność pojazdów, opinie, powiadomienia, dokumenty oraz panel administracyjny.

W implementacji wykorzystano rozwiązania oferowane przez Symfony, takie jak routing, formularze, walidacja oraz mechanizmy bezpieczeństwa. Doctrine ORM umożliwiło powiązanie obiektowego modelu aplikacji z relacyjną bazą MySQL, natomiast Twig został wykorzystany do przygotowania interfejsu użytkownika.

Aplikacja została następnie uruchomiona w środowisku produkcyjnym opartym na AWS EC2, nginx, PHP-FPM i MySQL.

W kolejnym rozdziale przedstawiono sposób testowania zaimplementowanych funkcji oraz wyniki przeprowadzonych testów.

4. Testowanie i wyniki

Testowanie stanowi istotny etap procesu tworzenia aplikacji internetowej, ponieważ umożliwia sprawdzenie poprawności działania zaimplementowanych funkcjonalności oraz wykrycie potencjalnych błędów przed udostępnieniem systemu użytkownikom. W przypadku aplikacji wspomagającej proces wynajmu samochodów szczególnie istotne jest zapewnienie poprawności operacji związanych z użytkownikami, pojazdami i rezerwacjami, a także właściwej kontroli dostępu do poszczególnych funkcji systemu.

W ramach realizacji projektu przeprowadzono testy automatyczne obejmujące najważniejsze elementy aplikacji. Do ich wykonania wykorzystano framework PHPUnit oraz narzędzia testowe dostępne w środowisku Symfony. Testy wykonywano na odseparowanym środowisku testowym, wykorzystującym osobną bazę danych.

Oprócz sprawdzenia poszczególnych funkcjonalności zweryfikowano również poprawność konfiguracji warstwy dostępu do danych oraz zgodność modelu encji ze strukturą bazy danych.

Cel i zakres testów

Podstawowym celem przeprowadzonych testów było sprawdzenie, czy zaimplementowany system realizuje wymagane funkcjonalności w sposób zgodny z założeniami projektu. Szczególną uwagę zwrócono na operacje mające wpływ na dane użytkowników oraz przebieg procesu wynajmu samochodu.

Testy obejmowały zarówno podstawowe operacje wykonywane przez użytkowników, jak i sytuacje związane z próbą uzyskania dostępu do zasobów bez odpowiednich uprawnień. Pozwoliło to na zweryfikowanie nie tylko poprawnego przebiegu standardowych scenariuszy, ale również reakcji systemu na działania niedozwolone.

Zakres przeprowadzonych testów obejmował między innymi:

- kontrolę dostępu do wybranych zasobów aplikacji,
- obsługę strony logowania oraz resetowania hasła,
- obsługę samochodów i ograniczenia dostępu do pojazdów innych użytkowników,
- proces rezerwacji pojazdów, w tym terminy zajęte i niepoprawne zakresy dat,
- akceptowanie i odrzucanie rezerwacji przez właściciela pojazdu,
- obsługę wniosków o zmianę terminu rezerwacji,
- system opinii i zabezpieczenie przed dodawaniem niedozwolonych opinii,
- system powiadomień,
- funkcje panelu administratora,
- rejestrowanie operacji administracyjnych,
- mechanizm ograniczający wielokrotne nieudane próby logowania.

Zastosowanie testów automatycznych umożliwiło wielokrotne wykonywanie tych samych scenariuszy po wprowadzaniu zmian w kodzie aplikacji. Dzięki temu możliwe było sprawdzenie, czy rozwój kolejnych funkcjonalności nie powodował regresji w istniejących elementach systemu.

Najważniejsze scenariusze testowe opisano w dalszych podrozdziałach, natomiast osobno ujęto krótkie zestawienie scenariuszy związanych z bezpieczeństwem i kontrolą dostępu. Dotyczą one ochrony zasobów oraz ograniczania operacji wykonywanych bez odpowiednich uprawnień.

Tabela 6. Scenariusze testów bezpieczeństwa i kontroli dostępu

ID	Scenariusz	Oczekiwany rezultat	Wynik
TS01	Użytkownik bez <code>ROLE_ADMIN</code> otwiera panel administratora	Brak dostępu	Pozytywny
TS02	Użytkownik próbuje usunąć cudzy samochód	Brak dostępu	Pozytywny
TS03	Żądanie zawiera błędny token CSRF	Operacja zostaje odrzucona	Pozytywny
TS04	Przesłanie niedozwolonego typu pliku	Błąd walidacji	Pozytywny
TS05	Próba wejścia do chronionej funkcji bez logowania	Brak dostępu lub przekierowanie do logowania	Pozytywny

Środowisko testowe

Testy aplikacji zostały wykonane w środowisku serwerowym, na którym uruchomiono projekt oparty na frameworku Symfony. Do przeprowadzania testów wykorzystano PHPUnit w wersji 9.6.22.

Testy funkcjonalne zostały zrealizowane między innymi z wykorzystaniem klasy `WebTestCase` dostarczanej przez Symfony. Mechanizm ten pozwala na uruchomienie jądra aplikacji oraz utworzenie klienta testowego symulującego działanie przeglądarki internetowej. Dzięki temu możliwe jest wykonywanie żądań HTTP do rzeczywistych tras aplikacji oraz sprawdzanie odpowiedzi generowanych przez kontrolery.

Istotnym elementem przygotowania środowiska było zastosowanie osobnej bazy danych przeznaczonej do testów. Baza testowa została oddzielona od bazy wykorzystywanej przez właściwą instancję aplikacji. Pozwoliło to na swobodne tworzenie, modyfikowanie i usuwanie danych testowych bez ryzyka naruszenia danych użytkowych systemu.

Przed uruchomieniem całego zestawu testów sprawdzono poprawność mapowania encji Doctrine oraz zgodność schematu bazy danych z aktualnym modelem aplikacji.

Wykorzystano w tym celu polecenie:

```
php bin/console doctrine:schema:validate --env=test
```

Wynikiem wykonania polecenia były komunikaty informujące, że pliki mapowania są poprawne, a schemat bazy danych jest zgodny z mapowaniem encji. Otrzymany rezultat potwierdził poprawność konfiguracji ORM oraz zgodność struktury testowej bazy danych z encjami wykorzystywanymi przez aplikację.

```
ubuntu@ip-172-31-29-150:/var/www/rental-app$ php bin/console doctrine:schema:validate --env=test
Mapping
-----
[OK] The mapping files are correct.

Database
-----
[OK] The database schema is in sync with the mapping files.

ubuntu@ip-172-31-29-150:/var/www/rental-app$ |
```

Rysunek 31. Wynik walidacji mapowania encji Doctrine i schematu bazy danych.

Cały zestaw testów uruchamiano za pomocą polecenia:

```
php bin/phpunit
```

Zastosowanie odrębnego środowiska testowego miało szczególne znaczenie w przypadku testów tworzących dane, ponieważ część scenariuszy wymagała utworzenia użytkowników, samochodów, rezerwacji, powiadomień oraz innych obiektów domenowych.

Organizacja testów automatycznych

Testy automatyczne zostały podzielone na osobne klasy odpowiadające głównym obszarom funkcjonalnym aplikacji. Takie rozwiązanie zwiększa czytelność kodu testowego oraz umożliwia uruchamianie testów konkretnego modułu niezależnie od całego zestawu.

Przykładowo testy dotyczące rezerwacji mogły zostać uruchomione poleceniem:

```
php bin/phpunit tests/Controller/ReservationControllerTest.php
```

natomiast pełny zestaw testów był wykonywany poleceniem:

```
php bin/phpunit
```

Tabela 7. Zestawienie testów automatycznych aplikacji

Klasa testowa	Testowany obszar	Liczba testów
AccessControllerTest	kontrola dostępu	5
AuthControllerTest	strona logowania	1
ResetPasswordControllerTest	resetowanie hasła	2
CarControllerTest	obsługa samochodów	9
ReservationControllerTest	obsługa rezerwacji	8
ReviewControllerTest	system opinii	6
NotificationControllerTest	system powiadomień	7
ReservationChangeRequestControllerTest	zmiana terminu rezerwacji	8
AdminControllerTest	funkcje administratora	9
LoginIpBlockerTest	zabezpieczenie logowania	4
AdminLogControllerTest	rejestrowanie działań administratora	4
Łącznie	11 klas testowych	63

Łącznie przygotowano 63 testy automatyczne rozmieszczone w 11 klasach testowych. Testy kontrolerów uporządkowano w katalogu `tests/Controller`, natomiast test mechanizmu ograniczającego wielokrotne próby logowania umieszczono w katalogu `tests/Security`.

Testy zawierały łącznie 153 asercje. Asercje służyły do sprawdzania między innymi kodów odpowiedzi HTTP, przekierowań, obecności określonych elementów w wygenerowanym dokumencie HTML oraz poprawności zachowania aplikacji po wykonaniu określonej operacji.

Testowanie poszczególnych modułów

Testy kontroli dostępu i logowania

Testy kontroli dostępu sprawdzały dostępność podstawowych stron aplikacji oraz reakcję systemu na próbę uzyskania dostępu do panelu administratora przez użytkownika niezalogowanego lub posiadającego jedynie standardową rolę. Dodatkowo osobny test weryfikował poprawne ładowanie strony logowania. Scenariusze te potwierdziły działanie podstawowego mechanizmu ochrony tras.

Testy mechanizmu resetowania hasła

Mechanizm resetowania hasła został sprawdzony poprzez test dostępności formularza oraz obsługi żądania resetowania dla istniejącego użytkownika. W ramach testu tworzony był użytkownik testowy, a następnie wykonywano żądanie prowadzące do formularza resetowania hasła i sprawdzano przekierowanie na stronę informującą o dalszym przebiegu procedury.

Testy obsługi samochodów

Testy modułu samochodów obejmowały dostęp do listy pojazdów, widoku szczegółów, formularza dodawania i edycji samochodu oraz reakcję systemu na próbę modyfikacji lub usunięcia pojazdu przez osobę niebędącą właścicielem. Zweryfikowano również obsługę nieistniejącego samochodu, dla którego aplikacja powinna zwrócić błąd 404.

Testy procesu rezerwacji

Proces rezerwacji samochodu został objęty osobnym zestawem testów, ponieważ stanowi jeden z głównych procesów biznesowych systemu. Sprawdzono między innymi blokadę rezerwacji własnego pojazdu, odrzucanie dat z przeszłości, kontrolę poprawności zakresu dat, brak możliwości rezerwacji niedostępnego auta oraz wykrywanie terminów nachodzących na istniejące rezerwacje. Testy obejmowały także akceptowanie i odrzucanie zgłoszeń przez właściciela pojazdu.

Testy zmiany terminu rezerwacji

Aplikacja umożliwia obsługę wniosków dotyczących zmiany terminu istniejącej rezerwacji. Testy sprawdzały, czy wniosek może zostać złożony wyłącznie przez właściwego użytkownika, czy obcy właściciel nie może obsłużyć cudzego wniosku oraz

czy niepoprawny token CSRF nie powoduje zmiany statusu. Zweryfikowano również metody encji odpowiedzialne za akceptację i odrzucenie wniosku.

Testy systemu opinii

Moduł opinii został sprawdzony pod kątem ograniczeń biznesowych i bezpieczeństwa. Testy potwierdziły, że właściciel nie może opiniować własnego auta, użytkownik bez zakończonej rezerwacji nie może dodać opinii, a ta sama osoba nie może dodać drugiej opinii dla tego samego pojazdu. Sprawdzone również zabezpieczenia usuwania opinii przed działaniem nieuprawnionego użytkownika oraz przed niepoprawnym tokenem CSRF.

Testy systemu powiadomień

System powiadomień odpowiada za przekazywanie użytkownikowi informacji dotyczących zdarzeń zachodzących w aplikacji. Testy sprawdzały dostęp do listy powiadomień, blokadę odczytu cudzych powiadomień, oznaczanie własnego powiadomienia jako przeczytanego oraz zabezpieczenie operacji oznaczania powiadomień niepoprawnym tokenem CSRF.

Testy panelu administratora

Panel administratora posiada funkcje, które nie powinny być dostępne dla standardowego użytkownika aplikacji. W testach zweryfikowano dostęp do pulpitu administratora, raportów i listy rezerwacji, a także reakcję systemu na próby wykonania operacji administracyjnych bez odpowiednich uprawnień lub z niepoprawnym tokenem CSRF.

Testy zabezpieczenia przed wielokrotnymi próbami logowania

Mechanizm ograniczający wielokrotne nieudane próby logowania został sprawdzony za pomocą testów jednostkowych klasy `LoginIpBlocker`. Testy potwierdziły początkowy brak blokady adresu IP, możliwość zablokowania adresu, usunięcia blokady oraz poprawną obsługę wartości pustej.

Testy rejestrowania działań administratora

Testy dziennika administracyjnego sprawdzały dostęp do logów administracyjnych dla różnych poziomów uprawnień. Zweryfikowano, że użytkownik niezalogowany jest przekierowywany do logowania, zwykły użytkownik oraz administrator bez roli superadministratora nie mają dostępu do logów, natomiast użytkownik z rolą `ROLE_SUPER_ADMIN` może otworzyć ten widok.

Wyniki przeprowadzonych testów

Po zweryfikowaniu poszczególnych modułów uruchomiono jednocześnie cały zestaw testów aplikacji. Pozwoliło to sprawdzić, czy wszystkie scenariusze przechodzą poprawnie również podczas jednego pełnego przebiegu PHPUnit.

Końcowy wynik przedstawiał się następująco:

```
OK (63 tests, 153 assertions)

ubuntu@ip-172-31-29-150:/var/www/rental-app$ php bin/phpunit
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Testing
..... 63 / 63 (100%)

Time: 00:05.864, Memory: 98.50 MB

OK (63 tests, 153 assertions)
ubuntu@ip-172-31-29-150:/var/www/rental-app$ |
```

Rysunek 32. Wynik wykonania pełnego zestawu testów automatycznych PHPUnit.

Tabela 8. Podsumowanie wyników testowania

Parametr	Wynik
Liczba klas testowych	11
Liczba testów	63
Liczba wykonanych asercji	153
Testy zakończone powodzeniem	63
Testy zakończone niepowodzeniem	0
Błędy wykonania testów	0
Poprawność mapowania Doctrine	poprawna
Zgodność schematu testowej bazy danych	poprawna
Wynik zestawu testowego	100% uruchomionych testów zakończonych powodzeniem

Wszystkie 63 testy zakończyły się powodzeniem. PHPUnit wykonał łącznie 153 asercje i nie zgłosił błędów ani niepowodzeń. Końcowy rezultat wskazuje, że wszystkie scenariusze objęte przygotowanym zestawem testowym działały zgodnie z oczekiwaniami w momencie wykonywania testów.

Problemy wykryte podczas testowania

Przeprowadzenie testów pozwoliło również wykryć problemy, które nie były bezpośrednio widoczne podczas standardowego korzystania z aplikacji. Jednym z nich była niepoprawna konfiguracja dwukierunkowej relacji pomiędzy encjami `User` oraz `Notification`. Walidator Doctrine wskazał, że strona odwrotna relacji odwoływała się do nieistniejącego odpowiednika w encji `Notification`. Po poprawieniu konfiguracji relacji ponownie wykonano walidację schematu i uzyskano poprawny wynik zarówno dla mapowania, jak i struktury bazy danych.

Podczas tworzenia testów konieczne było również odpowiednie przygotowanie i czyszczenie danych pomiędzy kolejnymi scenariuszami. Przykładem był mechanizm resetowania hasła. Powiązane rekordy mogły uniemożliwiać usunięcie użytkownika ze względu na ograniczenia klucza obcego. Problem rozwiązano poprzez zachowanie właściwej kolejności usuwania danych testowych.

Kolejnym zagadnieniem była obsługa sesji i tokenów CSRF podczas testowania operacji zmieniających stan rezerwacji. Operacje takie jak zaakceptowanie lub odrzucenie rezerwacji są chronione przed nieautoryzowanym wykonaniem. Testy musiały zatem odzwierciedlać rzeczywisty sposób działania aplikacji i uwzględniać wymagane mechanizmy bezpieczeństwa.

Podczas testowania pojawił się również problem związany z cyklem życia obiektów Doctrine. Po wykonaniu żądania HTTP część wcześniej utworzonych obiektów nie była już zarządzana przez aktualny `EntityManager`. Weryfikacja danych po wykonaniu operacji została więc oparta na ponownym pobraniu odpowiedniego rekordu z bazy danych. Rozwiązanie to pozwala sprawdzić rzeczywisty stan zapisany w bazie, zamiast polegać na stanie obiektu istniejącego wcześniej w pamięci aplikacji.

Znaczenie testów bezpieczeństwa i kontroli dostępu

Ze względu na charakter aplikacji szczególne znaczenie miała weryfikacja mechanizmów związanych z bezpieczeństwem i autoryzacją użytkowników. System przechowuje dane użytkowników oraz umożliwia wykonywanie operacji związanych z samochodami i rezerwacjami. Nieprawidłowo skonfigurowana kontrola dostępu mogłaby umożliwić użytkownikowi wykonanie operacji dotyczącej zasobu, do którego nie powinien posiadać uprawnień.

Testy obejmowały dlatego scenariusze dotyczące dostępu niezalogowanego użytkownika do chronionych funkcjonalności, dostępu standardowego użytkownika do panelu administratora, wykonywania operacji na cudzych zasobach, zabezpieczenia operacji za pomocą tokenów CSRF oraz ograniczania wielokrotnych prób logowania.

Należy jednocześnie zaznaczyć, że pozytywny wynik testów automatycznych nie oznacza całkowitego wyeliminowania wszystkich możliwych podatności bezpieczeństwa. Testy potwierdzają poprawność scenariuszy, które zostały nimi objęte.

Ocena rezultatów testowania

Uzyskane wyniki można uznać za pozytywne. Wszystkie przygotowane testy zakończyły się powodzeniem, a środowisko testowe wykazało zgodność modelu Doctrine ze strukturą bazy danych. Największą liczbą testów objęto obszary bezpośrednio związane z podstawową funkcjonalnością systemu, takie jak samochody, rezerwacje, zmiany terminów, powiadomienia i panel administracyjny.

Łączne wykonanie 63 testów i 153 asercji umożliwiło sprawdzenie wielu elementów aplikacji w sposób powtarzalny. Automatyzacja pozwala ponadto uruchamiać przygotowany zestaw ponownie po kolejnych zmianach w kodzie źródłowym.

Jednocześnie wyniku 63 / 63 (100%) nie należy interpretować jako stuprocentowego pokrycia całego kodu źródłowego testami. Informacja ta oznacza, że 100%

uruchomionych testów zakończyło się powodzeniem, a nie że testy obejmują 100% instrukcji lub wszystkich możliwych scenariuszy aplikacji.

Wnioski z testowania

Przeprowadzone testy automatyczne pozwoliły potwierdzić poprawność działania kluczowych funkcjonalności systemu wspomagającego proces wynajmu samochodów. Zweryfikowano zarówno podstawowe operacje biznesowe, jak i mechanizmy kontroli dostępu oraz wybrane elementy bezpieczeństwa.

W ramach końcowego przebiegu wykonano 63 testy zawierające 153 asercje. Wszystkie testy zakończyły się powodzeniem. Nie wystąpiły błędy wykonania ani nieudane asercje.

Testowanie przyczyniło się również do wykrycia i usunięcia problemów dotyczących między innymi konfiguracji relacji Doctrine, integralności danych testowych, obsługi sesji oraz cyklu życia encji. Pokazuje to znaczenie automatycznych testów nie tylko jako narzędzia końcowej weryfikacji aplikacji, lecz również jako elementu wspierającego proces jej implementacji.

Na podstawie otrzymanych rezultatów można stwierdzić, że funkcjonalności objęte przygotowanymi scenariuszami testowymi działają zgodnie z przyjętymi założeniami, a przygotowany zestaw testów stanowi podstawę do dalszej kontroli jakości rozwijanego systemu.

W ostatnim rozdziale przedstawiono podsumowanie wykonanej pracy, ocenę realizacji założonych celów oraz możliwe kierunki dalszego rozwoju systemu.

5. Podsumowanie i wnioski

Podsumowanie realizacji projektu

Celem niniejszej pracy było zaprojektowanie, implementacja oraz wdrożenie aplikacji webowej wspomagającej proces wynajmu pojazdów przez osoby prywatne. W ramach projektu opracowano system CarRental, który umożliwia obsługę najważniejszych etapów związanych z publikowaniem ofert samochodów, ich wyszukiwaniem, rezerwacją oraz późniejszą obsługą wynajmu.

Założony cel pracy został osiągnięty. Przygotowana aplikacja umożliwia użytkownikom utworzenie konta, zalogowanie się do systemu oraz zarządzanie własnym profilem. Użytkownik może przeglądać dostępne samochody, filtrować oferty według wybranych parametrów oraz przechodzić do szczegółowego widoku wybranego pojazdu.

System pozwala również użytkownikowi dodać własny samochód i tym samym występować w roli właściciela pojazdu udostępnianego innym osobom. Właściciel może zarządzać własnymi pojazdami oraz zgłoszeniami dotyczącymi ich wynajmu.

Dzięki temu jedno konto może być wykorzystywane zarówno do rezerwowania samochodów innych użytkowników, jak i do udostępniania własnych pojazdów.

Jednym z najważniejszych elementów aplikacji jest moduł rezerwacji. Umożliwia on powiązanie użytkownika z konkretnym pojazdem oraz określonym terminem wynajmu. System pozwala kontrolować dostępność samochodu, tworzyć rezerwacje oraz zarządzać ich statusami. Właściciel pojazdu może zaakceptować lub odrzucić zgłoszenie, dzięki czemu zachowuje bezpośrednią kontrolę nad udostępnieniem swojego samochodu.

W ramach projektu zaimplementowano także dodatkowe funkcje wspomagające proces wynajmu. Należą do nich między innymi obsługa zdjęć samochodów, opinie użytkowników, powiadomienia, generowanie dokumentów związanych z wynajmem oraz funkcje administracyjne. Poszczególne elementy systemu są powiązane poprzez wspólny model danych, co pozwala zachować spójność informacji dotyczących użytkowników, pojazdów i rezerwacji.

Aplikacja została również wdrożona na serwerze AWS EC2. Pozwoliło to zweryfikować jej działanie poza środowiskiem lokalnym oraz sprawdzić współpracę pomiędzy serwerem nginx, PHP-FPM, frameworkiem Symfony oraz bazą danych MySQL.

Ocena zastosowanych technologii

Do implementacji aplikacji wykorzystano język PHP oraz framework Symfony 6.4. Wybór Symfony okazał się odpowiedni dla realizowanego projektu, ponieważ framework zapewnia mechanizmy potrzebne do tworzenia aplikacji webowej o modularnej strukturze.

Routing pozwolił uporządkować adresy i przypisać je do odpowiednich kontrolerów. Formularze Symfony uprościły obsługę danych wprowadzanych przez użytkowników oraz umożliwiły zastosowanie walidacji. Komponent Security został wykorzystany do obsługi uwierzytelniania i ograniczania dostępu do chronionych elementów aplikacji.

Doctrine ORM pozwoliło odwzorować model danych za pomocą klas PHP oraz relacji pomiędzy encjami. Dzięki temu możliwe było powiązanie użytkowników, samochodów, rezerwacji, opinii i pozostałych elementów systemu bez konieczności ręcznego tworzenia wszystkich podstawowych zapytań SQL.

Do przechowywania danych wykorzystano bazę MySQL. Relacyjny charakter bazy dobrze odpowiada strukturze systemu, w którym występują wyraźne zależności pomiędzy użytkownikami, samochodami i rezerwacjami.

Warstwę prezentacji przygotowano w Twig, co umożliwiło oddzielenie generowania widoków od głównej logiki aplikacji. Bootstrap został wykorzystany do przygotowania podstawowego układu stron oraz elementów interfejsu.

Wdrożenie na AWS EC2 pozwoliło zdobyć dodatkowe doświadczenie związane z konfiguracją środowiska produkcyjnego, serwera nginx, PHP-FPM oraz bazy danych. Etap ten pokazał również znaczenie właściwego przechowywania konfiguracji i danych wrażliwych poza repozytorium kodu źródłowego.

Wnioski wynikające z realizacji projektu

Realizacja projektu pozwoliła zauważyć, że wykonanie aplikacji obsługującej proces wynajmu wymaga rozwiązania znacznie większej liczby problemów niż samo przygotowanie interfejsu użytkownika.

Jednym z najważniejszych zagadnień okazało się poprawne zaprojektowanie modelu danych. Informacje dotyczące użytkowników, pojazdów i rezerwacji są ze sobą bezpośrednio powiązane, dlatego błędy na tym etapie mogą znacząco utrudnić dalszą rozbudowę systemu. Wykorzystanie relacji Doctrine pozwoliło uporządkować zależności pomiędzy najważniejszymi encjami.

Istotnym elementem była również kontrola uprawnień. W aplikacji nie wystarczy sprawdzić, czy użytkownik jest zalogowany. W wielu sytuacjach konieczne jest również ustalenie, czy dana osoba jest właścicielem konkretnego zasobu. Przykładem jest możliwość zarządzania samochodem lub rezerwacją dotyczącą własnego pojazdu.

Kolejnym ważnym wnioskiem jest znaczenie kontroli terminów. W procesie wynajmu samochodu podstawowym warunkiem poprawności jest niedopuszczenie do sytuacji, w której ten sam pojazd zostanie udostępniony kilku osobom w nakładających się okresach. Mechanizm sprawdzania dostępności jest więc jednym z kluczowych elementów logiki biznesowej systemu.

Duże znaczenie ma również poprawna konfiguracja środowiska produkcyjnego. Aplikacja działająca poprawnie na komputerze programisty nie musi automatycznie działać identycznie po wdrożeniu. Podczas pracy nad projektem konieczne było rozwiązanie problemów związanych między innymi z konfiguracją bazy danych, ścieżkami do przesyłanych plików, nazwami tras oraz ustawieniami środowiska Symfony.

Realizacja projektu potwierdziła także znaczenie odpowiedniego zarządzania danymi konfiguracyjnymi. Hasła, klucze oraz inne informacje wrażliwe nie powinny być przechowywane bezpośrednio w repozytorium kodu źródłowego. Przeniesienie ich do konfiguracji środowiskowej stanowi istotny element przygotowania aplikacji do działania na serwerze produkcyjnym.

Ograniczenia aktualnej wersji systemu

Mimo zrealizowania podstawowych założeń CarRental posiada również ograniczenia wynikające z zakresu pracy oraz przyjętych założeń projektowych.

Jednym z najważniejszych ograniczeń jest brak bezpośredniej obsługi płatności elektronicznych. System może przechowywać informacje potrzebne do obsługi rezerwacji i dokumentów, jednak samo rozliczenie pomiędzy stronami odbywa się poza aplikacją. Wprowadzenie płatności wymagałoby dodatkowej analizy prawnej, wyboru operatora płatności, obsługi potwierdzeń transakcji, zwrotów, reklamacji, zabezpieczenia przepływu danych oraz dostosowania systemu do wymagań dostawcy usług płatniczych.

Aktualna wersja systemu nie posiada również rozbudowanego mechanizmu weryfikacji tożsamości użytkownika. W przypadku rzeczywistego komercyjnego wykorzystania platformy należałoby rozważyć dodatkowe rozwiązania pozwalające na potwierdzenie danych właściciela oraz osoby wynajmującej samochód.

System nie wykorzystuje również rozwiązań telematycznych instalowanych w pojeździe. Oznacza to, że aplikacja nie umożliwia na przykład zdalnego otwierania samochodu, pobierania jego aktualnej lokalizacji ani automatycznego odczytywania parametrów pojazdu.

Ograniczeniem jest także zakres testów automatycznych. Podstawowe mechanizmy zostały sprawdzone podczas testowania aplikacji, jednak rozbudowanie zestawu testów integracyjnych oraz end-to-end zwiększyłoby odporność projektu na błędy pojawiające się podczas dalszego rozwoju.

Możliwości dalszego rozwoju

Struktura projektu pozwala na dalsze rozszerzanie aplikacji o dodatkowe funkcje.

Jednym z możliwych kierunków jest integracja z operatorem płatności elektronicznych. System mógłby umożliwiać opłacenie rezerwacji, pobranie zaliczki lub kaucji oraz automatyczne aktualizowanie statusu płatności. Taki moduł powinien jednak zostać zaprojektowany jako oddzielna część systemu, z uwzględnieniem webhooków operatora, historii transakcji, obsługi błędów, zwrotów oraz zasad bezpieczeństwa wymaganych przy przetwarzaniu danych finansowych.

Kolejną możliwością jest rozbudowanie mechanizmu weryfikacji użytkowników. Można byłoby wprowadzić między innymi weryfikację prawa jazdy, dodatkowe potwierdzenie danych osobowych lub integrację z zewnętrzną usługą umożliwiającą potwierdzenie tożsamości.

System mógłby zostać również rozszerzony o bardziej rozbudowany kalendarz dostępności. Właściciel mógłby samodzielnie oznaczać dni, w których samochód nie jest dostępny, niezależnie od istniejących rezerwacji.

Kolejnym kierunkiem rozwoju jest mechanizm automatycznych przypomnień. Użytkownik mógłby otrzymywać wiadomość e-mail lub SMS przed rozpoczęciem wynajmu, przed terminem zwrotu pojazdu albo po zmianie statusu rezerwacji.

Możliwe jest również rozszerzenie panelu administratora o bardziej szczegółowe raporty i statystyki, na przykład liczbę rezerwacji w poszczególnych miesiącach, najczęściej wynajmowane samochody, przychody wynikające z rezerwacji lub aktywność poszczególnych użytkowników.

W przyszłości warto również rozważyć rozwój warstwy API. Pozwoliłoby to na utworzenie oddzielnej aplikacji mobilnej wykorzystującej tę samą logikę i bazę danych co wersja webowa.

Kolejnym etapem mogłaby być automatyzacja procesu wdrażania aplikacji. Zastosowanie mechanizmu CI/CD pozwoliłoby wykonywać testy po każdej zmianie kodu, a po ich poprawnym zakończeniu automatycznie wdrażać zatwierdzoną wersję na serwer.

Rozbudowy wymagać może również system wykonywania kopii zapasowych oraz monitorowania działania aplikacji. W przypadku rzeczywistej platformy wykorzystywanej przez większą liczbę użytkowników istotne byłoby zapewnienie regularnych kopii bazy danych, monitorowania dostępności serwera oraz rejestrowania istotnych zdarzeń technicznych.

Wnioski końcowe

W ramach niniejszej pracy udało się zaprojektować, zaimplementować i wdrożyć działającą aplikację webową wspomagającą proces wynajmu pojazdów przez osoby prywatne.

CarRental realizuje podstawowy scenariusz zakładany na początku pracy. Użytkownik może utworzyć konto, wyszukać samochód, dokonać rezerwacji oraz śledzić jej status. Jednocześnie może udostępnić własny pojazd i zarządzać zgłoszeniami dotyczącymi jego wynajmu. System integruje te funkcje z obsługą dokumentów, opinii, powiadomień i administracji.

Realizacja projektu pozwoliła zastosować w praktyce wiedzę dotyczącą projektowania aplikacji webowych, programowania w PHP, frameworka Symfony, relacyjnych baz danych, bezpieczeństwa oraz wdrażania aplikacji na serwerze.

Projekt pokazuje również, że podstawowy proces wynajmu pojazdu pomiędzy osobami prywatnymi może zostać obsłużony w ramach jednej aplikacji internetowej, bez konieczności wykorzystywania wielu niezależnych narzędzi do przechowywania informacji o samochodach, rezerwacjach i dokumentach.

Jednocześnie obecna wersja stanowi podstawę do dalszego rozwoju. Dodanie płatności elektronicznych, bardziej rozbudowanej weryfikacji użytkowników, automatycznych powiadomień, raportowania oraz dodatkowych mechanizmów bezpieczeństwa mogłoby stopniowo przekształcić projekt w bardziej kompleksową platformę wspierającą rzeczywisty proces wynajmu pojazdów pomiędzy użytkownikami.

Bibliografia

1. Fowler M., Patterns of Enterprise Application Architecture, Addison-Wesley, Boston 2002.
2. Pressman R. S., Maxim B. R., Software Engineering: A Practitioner's Approach, McGraw-Hill Education, New York 2019.
3. Sommerville I., Software Engineering, wyd. 10, Pearson, Boston 2015.
4. Welling L., Thomson L., PHP and MySQL Web Development, wyd. 5, Addison-Wesley, Boston 2016.

Linkografia

1. Dokumentacja frameworku Symfony 6.4, <https://symfony.com/doc/6.4/index.html>
2. Dokumentacja Doctrine ORM, <https://www.doctrine-project.org/projects/orm.html>
3. Dokumentacja Twig, <https://twig.symfony.com/doc/>
4. Dokumentacja języka PHP, <https://www.php.net/manual/en/>
5. Dokumentacja bazy danych MySQL, <https://dev.mysql.com/doc/>
6. Dokumentacja frameworka Bootstrap, <https://getbootstrap.com/docs/>
7. Dokumentacja PHPUnit, <https://phpunit.de/documentation.html>
8. Dokumentacja Composer, <https://getcomposer.org/doc/>
9. Dokumentacja HitHub, <https://docs.github.com/>
10. Dokumentacja AWS EC2, <https://docs.aws.amazon.com/ec2/>
11. Dokumentacja nginx, <https://nginx.org/en/docs/>
14. Turo, <https://turo.com>
15. Getaround, <https://getaround.com>
17. HoppyGo, <https://hoppygomobility.com>

Spis rysunków

Rysunek 1. Mapa przypadków użycia systemu CarRental.	16
Rysunek 2. Diagram procesu rezerwacji pojazdu.	17
Rysunek 3. Diagram stanów rezerwacji w systemie CarRental.	17
Rysunek 4. Diagram architektury logicznej systemu CarRental.	18
Rysunek 5. Diagram ERD modelu danych aplikacji CarRental.	19
Rysunek 6. Formularz rejestracji użytkownika w aplikacji CarRental.	22
Rysunek 7. Ekran logowania użytkownika.	23

Rysunek 8. Widok profilu użytkownika.	23
Rysunek 9. Formularz edycji profilu użytkownika.	24
Rysunek 10. Widok listy samochodów dodanych przez użytkownika.	25
Rysunek 11. Formularz dodawania samochodu - podstawowe dane pojazdu.	25
Rysunek 12. Formularz dodawania samochodu - dane techniczne, zdjęcia i zapis ogłoszenia.	26
Rysunek 13. Formularz edycji samochodu - podstawowe dane i dostępność.	27
Rysunek 14. Formularz edycji samochodu - zarządzanie zdjęciami pojazdu.	27
Rysunek 15. Polecane auta prezentowane na stronie głównej aplikacji.	28
Rysunek 16. Widok szczegółów samochodu z kalendarzem rezerwacji.	28
Rysunek 17. Lista rezerwacji obsługiwanych przez użytkownika udostępniającego pojazd.	30
Rysunek 18. Potwierdzenie rezerwacji wraz z podsumowaniem kosztów.	30
Rysunek 19. Widok rezerwacji przypisanych do zalogowanego użytkownika.	31
Rysunek 20. Widok szczegółów rezerwacji z aktualnym statusem i dostępnymi akcjami.	33
Rysunek 21. Sekcja opinii użytkowników na stronie pojazdu.	33
Rysunek 22. Lista powiadomień użytkownika.	34
Rysunek 23. Przykład faktury VAT wygenerowanej na podstawie danych rezerwacji.	36
Rysunek 24. Przykład protokołu wydania i zwrotu pojazdu wygenerowanego przez system.	38
Rysunek 25. Panel administratora - zarządzanie użytkownikami, rolami i blokadami kont.	39
Rysunek 26. Główny panel administratora systemu CarRental.	39
Rysunek 27. Panel administratora - lista rezerwacji z filtrowaniem i statusem.	40
Rysunek 28. Przykład responsywnego widoku aplikacji na ekranie o mniejszej szerokości.	42
Rysunek 29. Strona główna aplikacji CarRental.	43
Rysunek 30. Diagram wdrożenia aplikacji w środowisku AWS EC2.	44
Rysunek 31. Wynik walidacji mapowania encji Doctrine i schematu bazy danych.	49
Rysunek 32. Wynik wykonania pełnego zestawu testów automatycznych PHPUnit.	52

Spis tabel

Tabela 1. Wymagania funkcjonalne systemu CarRental	9
Tabela 2. Wymagania нефункционалне systemu CarRental	10
Tabela 3. Porównanie wybranych platform wynajmu pojazdów z systemem CarRental	12
Tabela 4. Technologie wykorzystane podczas implementacji aplikacji.....	21
Tabela 5. Zestawienie problemów napotkanych podczas implementacji i wdrożenia ...	46
Tabela 6. Scenariusze testów bezpieczeństwa i kontroli dostępu.....	48
Tabela 7. Zestawienie testów automatycznych aplikacji.....	49
Tabela 8. Podsumowanie wyników testowania	52

Załączniki

Załącznik A. Nośnik CD - zawartość

Zawartość nośnika CD - CarRental:

Pliki projektu CarRental:

- kod źródłowy aplikacji internetowej CarRental,
- pliki konfiguracyjne frameworka Symfony,
- szablony widoków Twig,
- migracje bazy danych,
- testy automatyczne aplikacji,
- pliki określające zależności projektu: composer.json, composer.lock, package.json oraz package-lock.json.

Dodatkowe materiały wykorzystane do przygotowania pracy:

- diagramy projektowe systemu,
- zrzuty ekranu najważniejszych widoków aplikacji,
- zrzuty ekranu przedstawiające wyniki testów,
- przykładowe dokumenty generowane przez system, w tym faktura VAT oraz protokół wydania i zwrotu pojazdu.

Dokumentacja i źródła:

- inżynierska praca dyplomowa w wersji elektronicznej w formacie krystian_gornowicz_50530.docx oraz krystian_gornowicz_50530.pdf,
- paczka z kodem źródłowym aplikacji CarRental,
- repozytorium projektu: <https://github.com/gornowicz/rental-app>,
- instrukcja uruchomienia projektu zawarta w pliku README_ODDANIE.md,
- plik .env.example z przykładową konfiguracją środowiska.